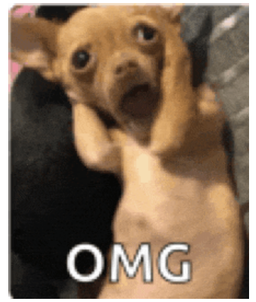


Transformer

Nguyễn Trần Thiên An
Đinh Xuân Khương



Attention Is All You Need!

Table of contents

01

RNN & LSTM

02

Overall of Tranformers

03

Token Embeddings + Positional encoding

04

Self-attention + multi-head attention

05

Residual Connection

06

Layer Normalization

07

FFN

08

Masked + Cross attention

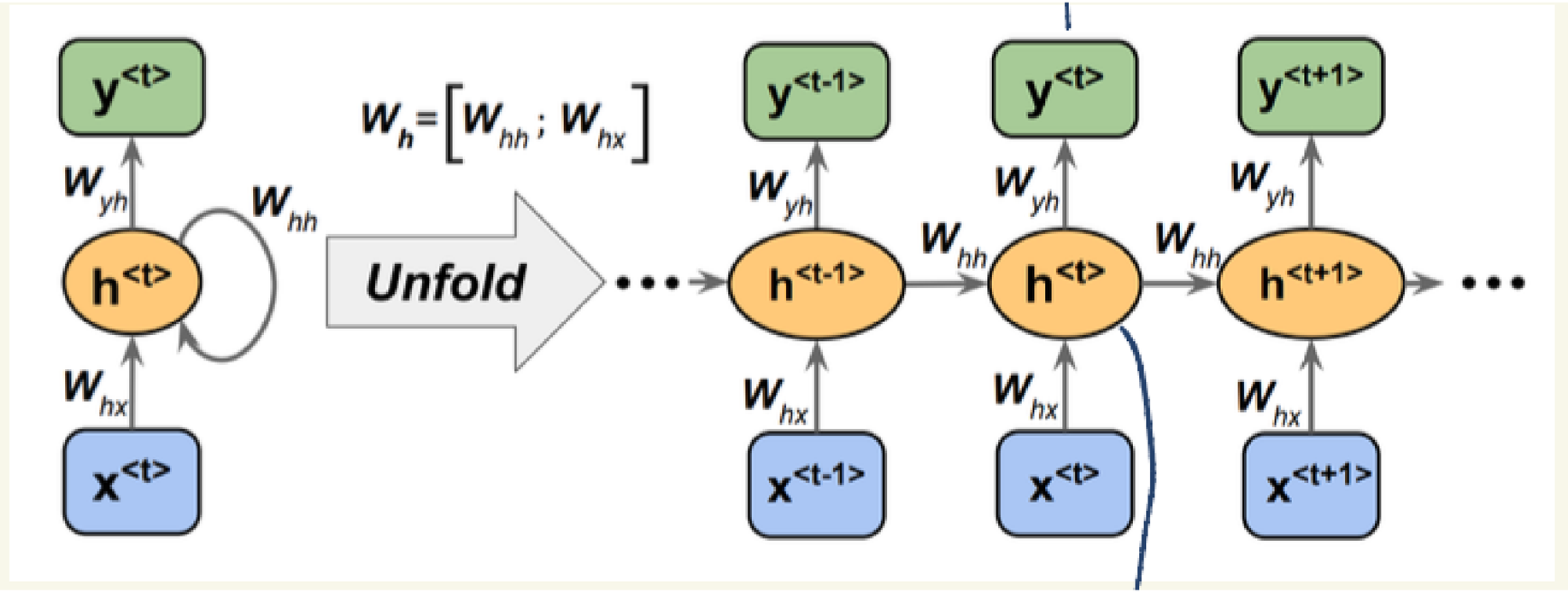
09

Temperature

01

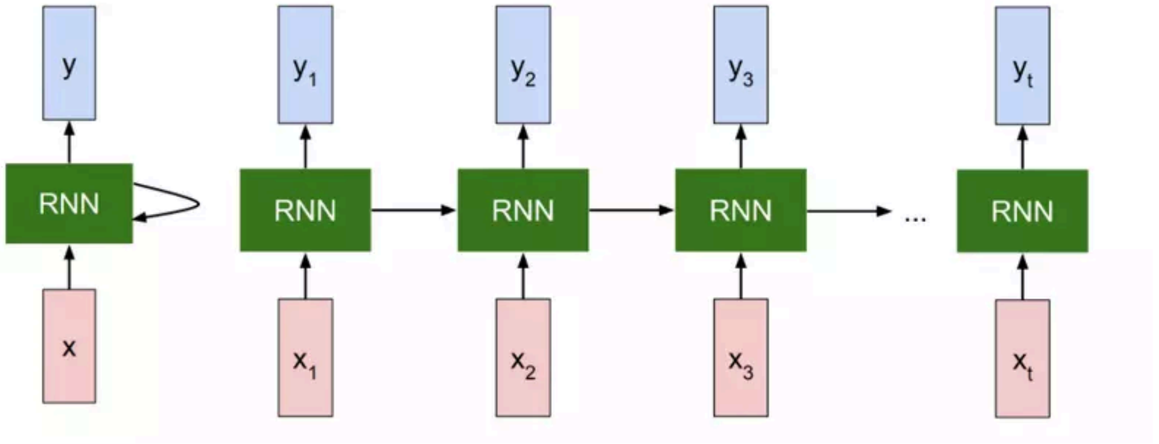
Problem of RNN & LSTM

Recurrent Neural Network (RNN)

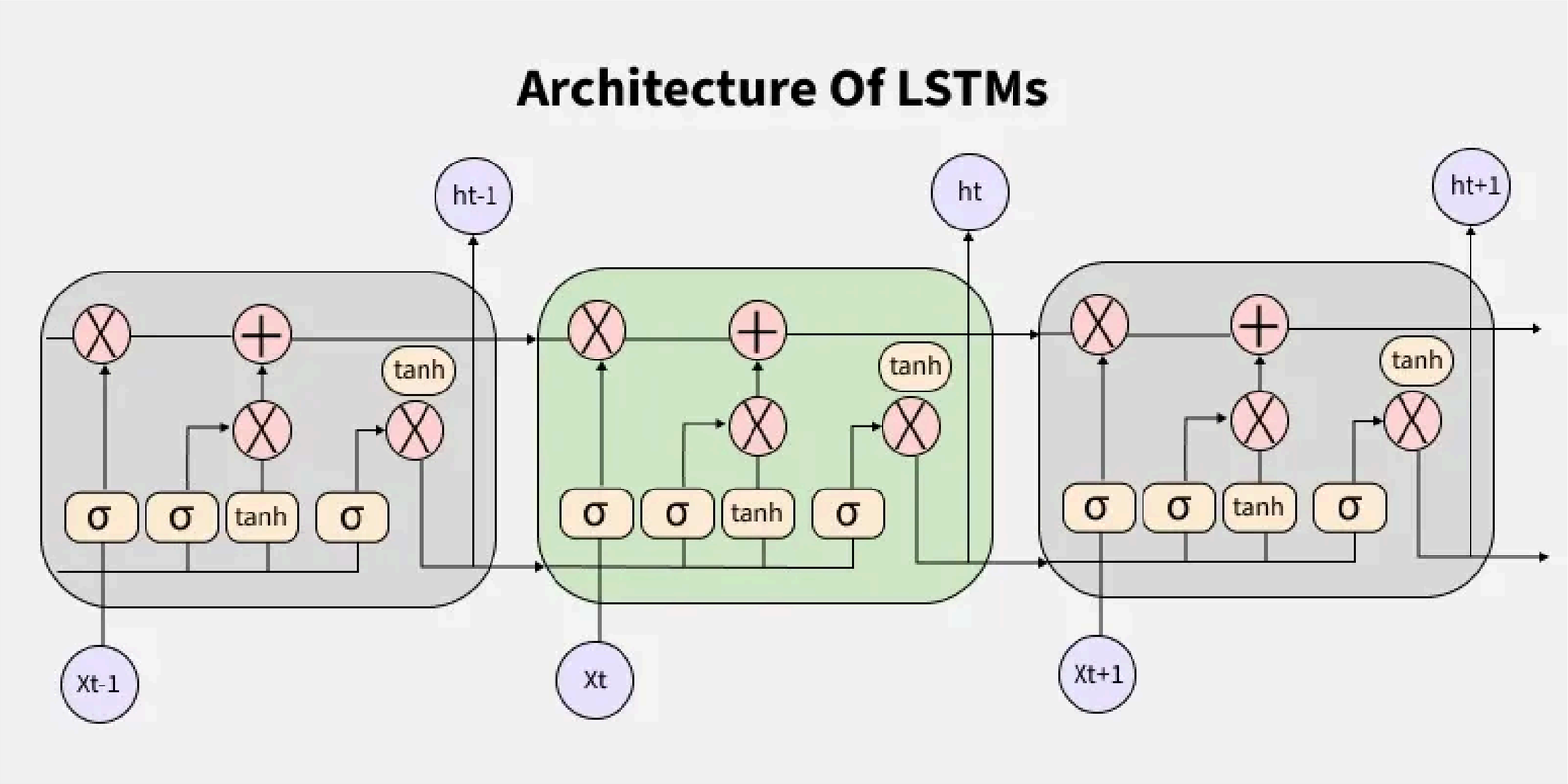


Problem of RNN

1. Sequential Computation (Cannot take advantage of the parallel computing capabilities of the GPU/TPU)
2. Vanishing Gradient Problem
3. Difficulty Learning Long-Term Dependencies



Long Short-Term Memory



02

Overview of Tranformers

Encoder

Prediction Head

Decoder

Feed Forward Layer

Layer Normalization

Multi-Head Attention!
(list of parallel self-attentions)

Residual Connection

Multi-Head Attention!
(list of parallel cross-attentions)

Masked Multi-Head Attention!
(list of parallel masked-attentions)

Embeddings
(Token and Positional)

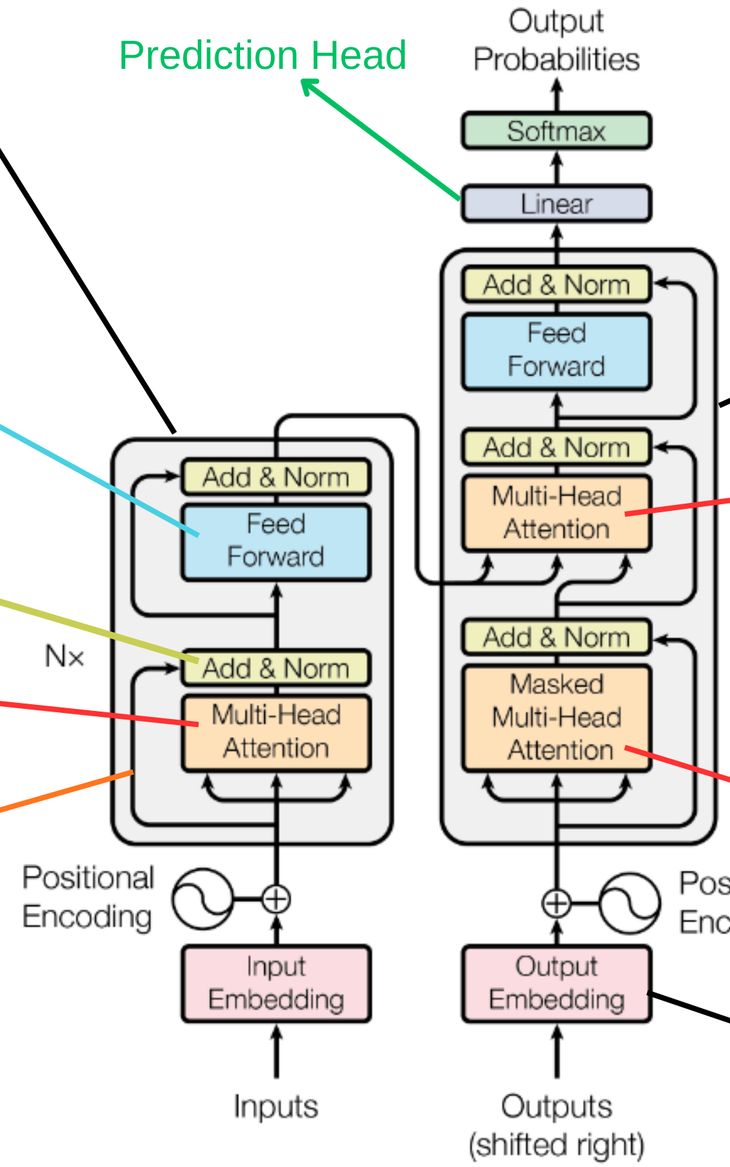


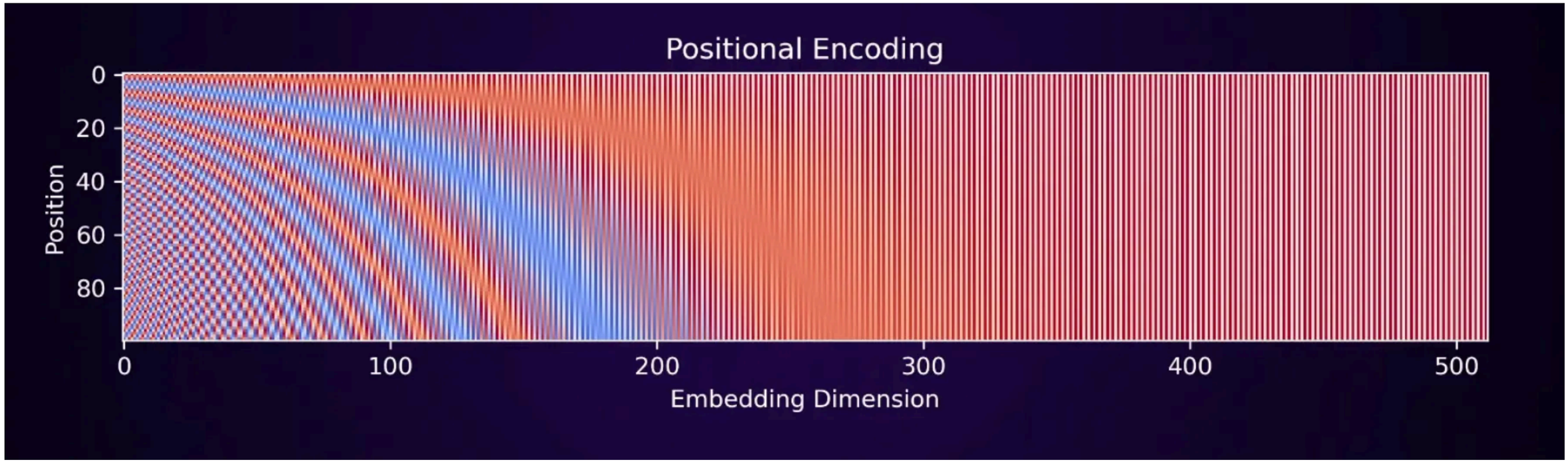
Figure 1: The Transformer - model architecture.

Nx: number of continously connected encoders/decoders

03

Positional Encoding

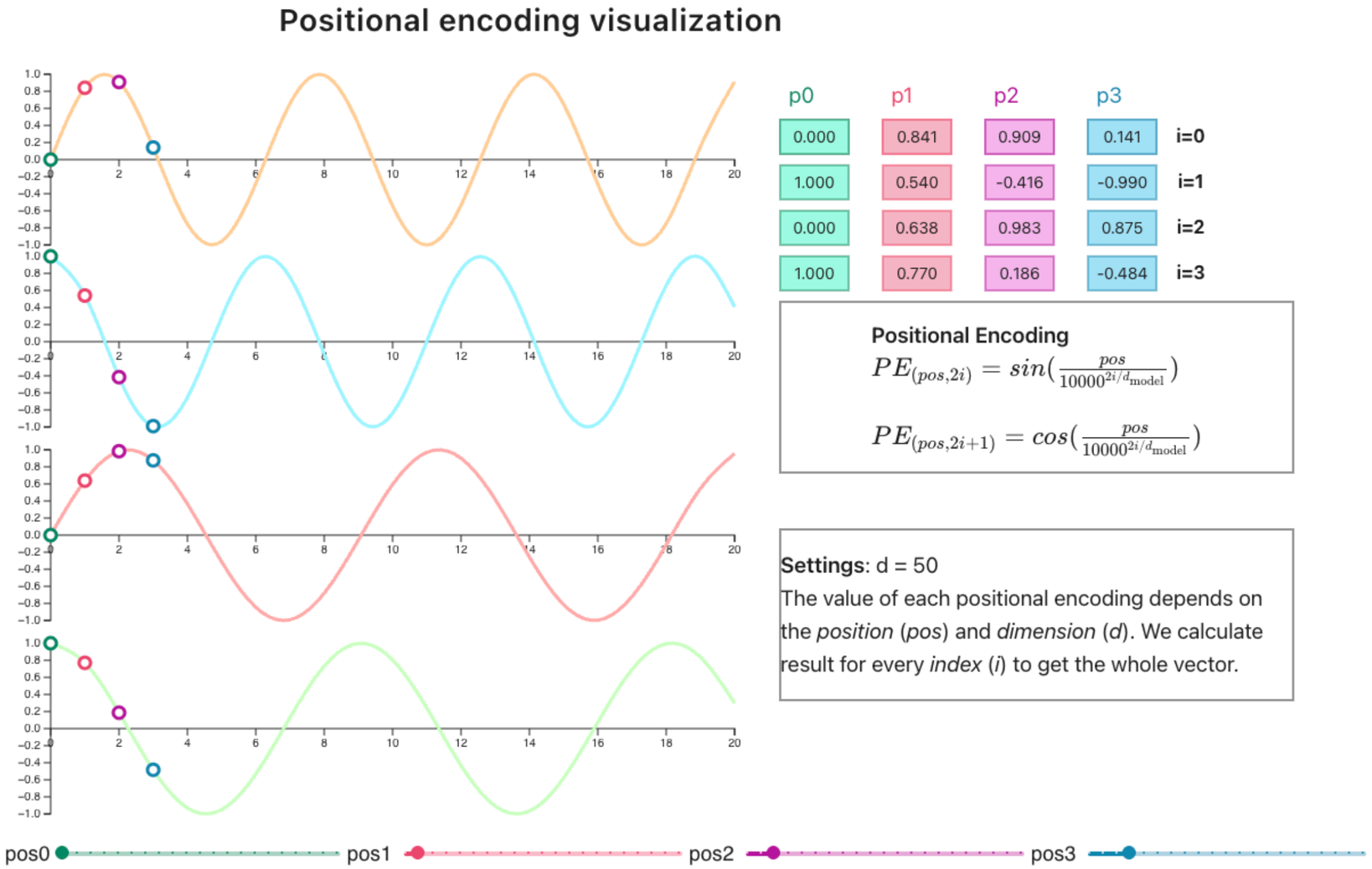
Can we understand this?



Properties

1. Unambiguous (position <=> value)
2. Deterministic (based on some underlying rule)
3. Allows to estimate distance between two token’s positions
4. Works with sequences longer than encountered before.

Sinusoidal Positional Encoding



	p0	p1	p2	p3	
	0.000	0.841	0.909	0.141	i=0
	1.000	0.540	-0.416	-0.990	i=1
	0.000	0.638	0.983	0.875	i=2
	1.000	0.770	0.186	-0.484	i=3

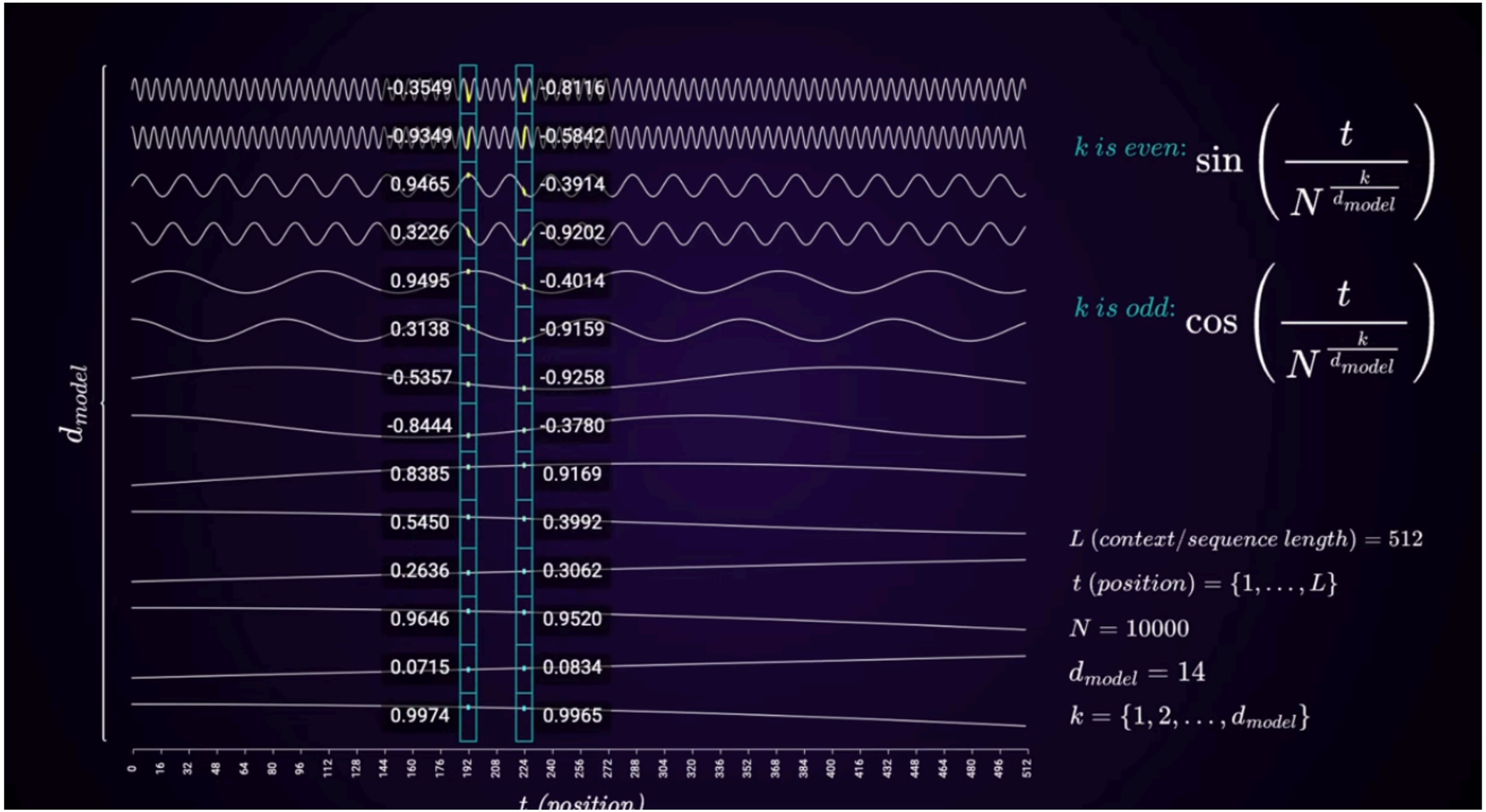
Positional Encoding

$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

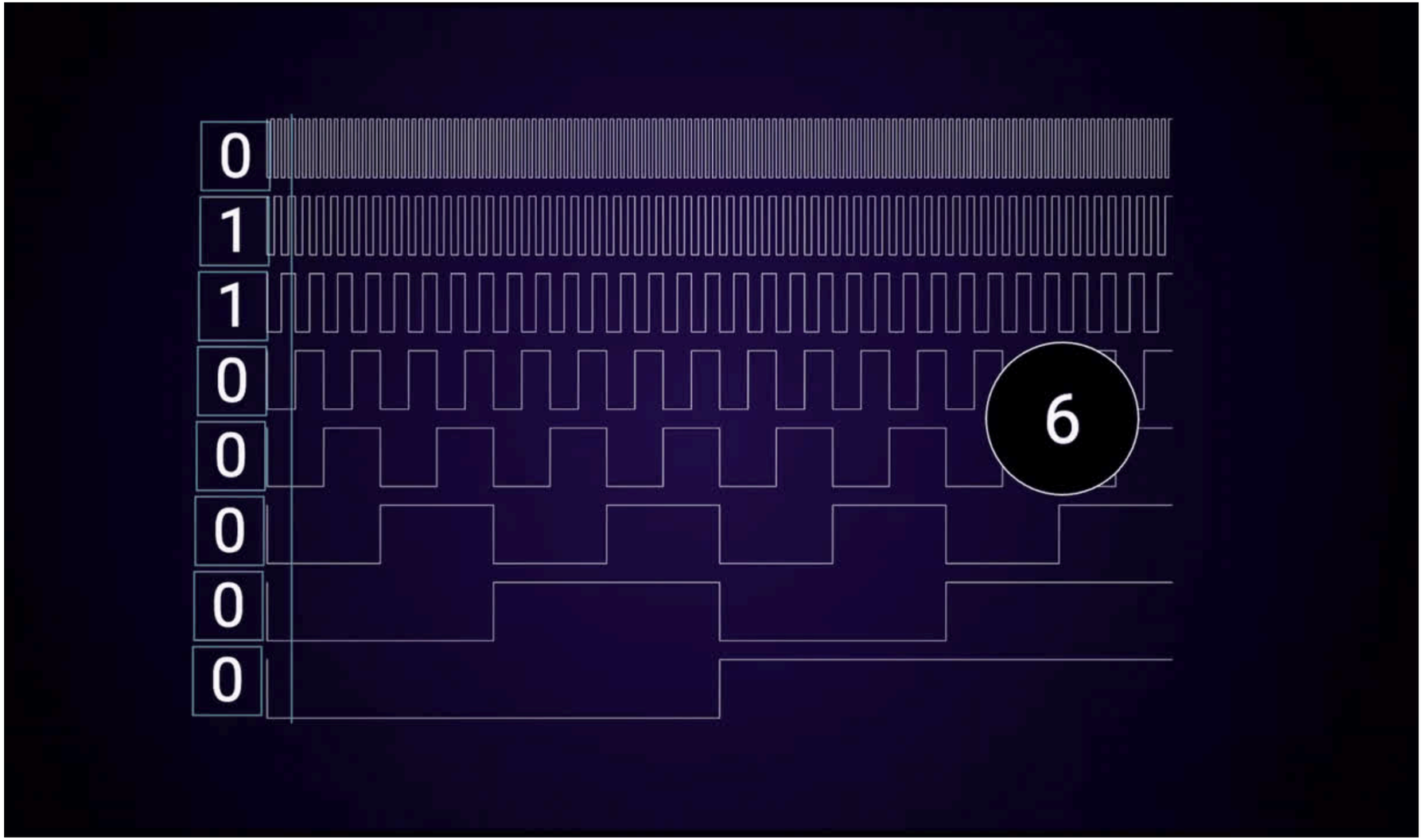
$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

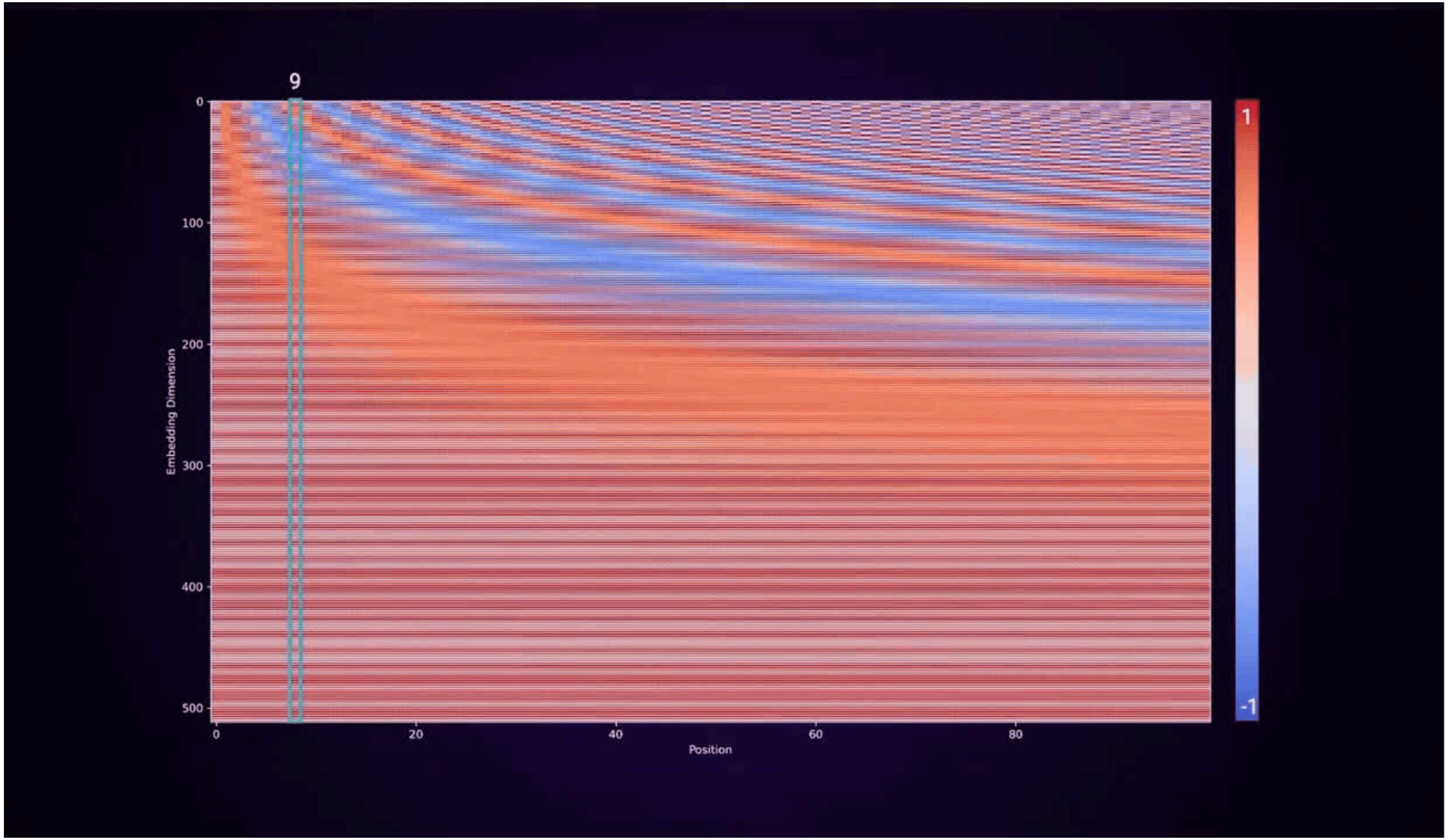
Settings: d = 50

The value of each positional encoding depends on the *position* (*pos*) and *dimension* (*d*). We calculate result for every *index* (*i*) to get the whole vector.


-0.3549
-0.8116
-0.9349
-0.5842
0.9465
-0.3914
0.3226
-0.9202
0.9495
-0.4014
0.3138
-0.9159
-0.5357
-0.9258
-0.8444
-0.3780
0.8385
0.9169
0.5450
0.3992
0.2636
0.3062
0.9646
0.9520
0.0715
0.0834
0.9974
0.9965
d_{model}
0 16 32 48 64 80 96 112 128 144 160 176 192 208 224 240 256 272 288 304 320 336 352 368 384 400 416 432 448 464 480 496 512
t (position)
k is even: $\sin\left(\frac{t}{N^{\frac{k}{d_{model}}}}\right)$
k is odd: $\cos\left(\frac{t}{N^{\frac{k}{d_{model}}}}\right)$
L (context/sequence length) $= 512$
t (position) $= \{1, \ldots, L\}$
$N = 10000$
$d_{model} = 14$
$k = \{1, 2, \ldots, d_{model}\}$

Binary Encoding




9
0
100
200
300
400
500
Embedding Dimension
0
20
40
60
80
Position
1
-1

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

$$\vec{v}_t = \begin{bmatrix} \sin(t) \\ \cos(t) \\ \sin\left(\frac{t}{\eta_1}\right) \\ \cos\left(\frac{t}{\eta_1}\right) \\ \vdots \\ \vdots \\ \sin\left(\frac{t}{\eta_{\frac{k}{2}}}\right) \\ \cos\left(\frac{t}{\eta_{\frac{k}{2}}}\right) \end{bmatrix} \text{ and } \vec{v}_{t+\delta} = \begin{bmatrix} \sin(t+\delta) \\ \cos(t+\delta) \\ \sin\left(\frac{t+\delta}{\eta_1}\right) \\ \cos\left(\frac{t+\delta}{\eta_1}\right) \\ \vdots \\ \vdots \\ \sin\left(\frac{t+\delta}{\eta_{\frac{k}{2}}}\right) \\ \cos\left(\frac{t+\delta}{\eta_{\frac{k}{2}}}\right) \end{bmatrix}$$

$$\vec{v}_{t+\delta} = \begin{bmatrix} \sin(t+\delta) \\ \cos(t+\delta) \\ \sin\left(\frac{t+\delta}{\eta_1}\right) \\ \cos\left(\frac{t+\delta}{\eta_1}\right) \\ \vdots \\ \vdots \\ \sin\left(\frac{t+\delta}{\eta_{\frac{k}{2}}}\right) \\ \cos\left(\frac{t+\delta}{\eta_{\frac{k}{2}}}\right) \end{bmatrix} = \begin{bmatrix} \sin(t)\cos(\delta) + \cos(t)\sin(\delta) \\ \cos(t)\cos(\delta) - \sin(t)\sin(\delta) \\ \sin\left(\frac{t}{\eta_1}\right)\cos\left(\frac{\delta}{\eta_1}\right) + \cos\left(\frac{t}{\eta_1}\right)\sin\left(\frac{\delta}{\eta_1}\right) \\ \cos\left(\frac{t}{\eta_1}\right)\cos\left(\frac{\delta}{\eta_1}\right) - \sin\left(\frac{t}{\eta_1}\right)\sin\left(\frac{\delta}{\eta_1}\right) \\ \vdots \\ \vdots \\ \sin\left(\frac{t}{\eta_{\frac{k}{2}}}\right)\cos\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) + \cos\left(\frac{t}{\eta_{\frac{k}{2}}}\right)\sin\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) \\ \cos\left(\frac{t}{\eta_{\frac{k}{2}}}\right)\cos\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) - \sin\left(\frac{t}{\eta_{\frac{k}{2}}}\right)\sin\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) \end{bmatrix}$$

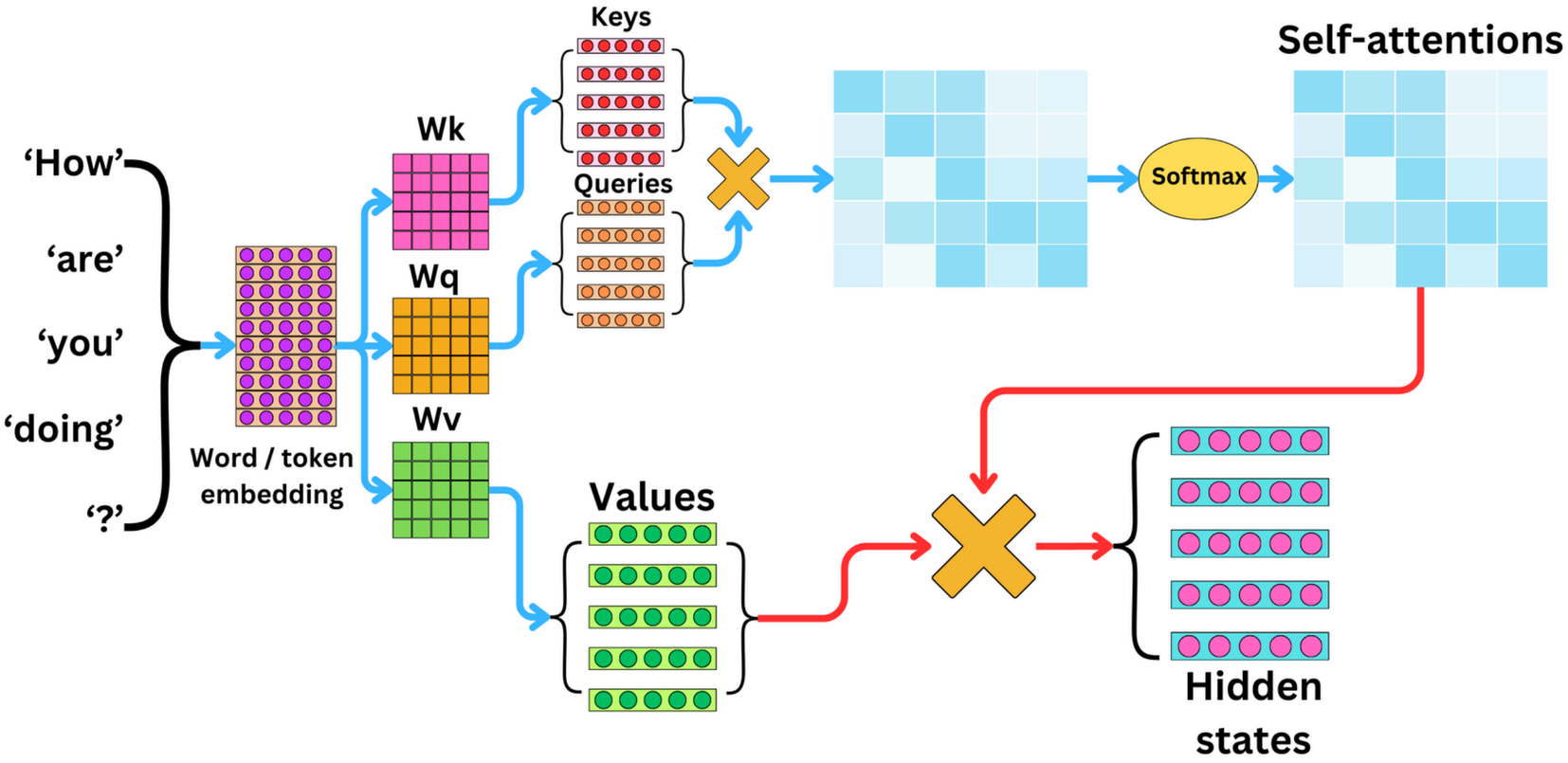
$$\sin(\alpha+\beta) = \sin(\alpha)\cos(\beta) + \cos(\alpha)\sin(\beta)$$
$$\cos(\alpha+\beta) = \cos(\alpha)\cos(\beta) - \sin(\alpha)\sin(\beta)$$

$$= \begin{bmatrix} \cos(\delta) & \sin(\delta) & & & & \\ -\sin(\delta) & \cos(\delta) & & & & \\ & & \cos\left(\frac{\delta}{\eta_1}\right) & \sin\left(\frac{\delta}{\eta_1}\right) & & \\ & & -\sin\left(\frac{\delta}{\eta_1}\right) & \cos\left(\frac{\delta}{\eta_1}\right) & & \\ & & & & \ddots & \\ & & & & \cos\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) & \sin\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) \\ & & & & -\sin\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) & \cos\left(\frac{\delta}{\eta_{\frac{k}{2}}}\right) \end{bmatrix} \times \begin{bmatrix} \sin(t) \\ \cos(t) \\ \sin\left(\frac{t}{\eta_1}\right) \\ \cos\left(\frac{t}{\eta_1}\right) \\ \vdots \\ \vdots \\ \sin\left(\frac{t}{\eta_{\frac{k}{2}}}\right) \\ \cos\left(\frac{t}{\eta_{\frac{k}{2}}}\right) \end{bmatrix}$$

04

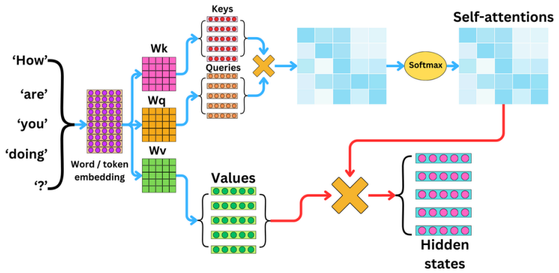
Self Attention & Multi-head Attention

Self-Attention

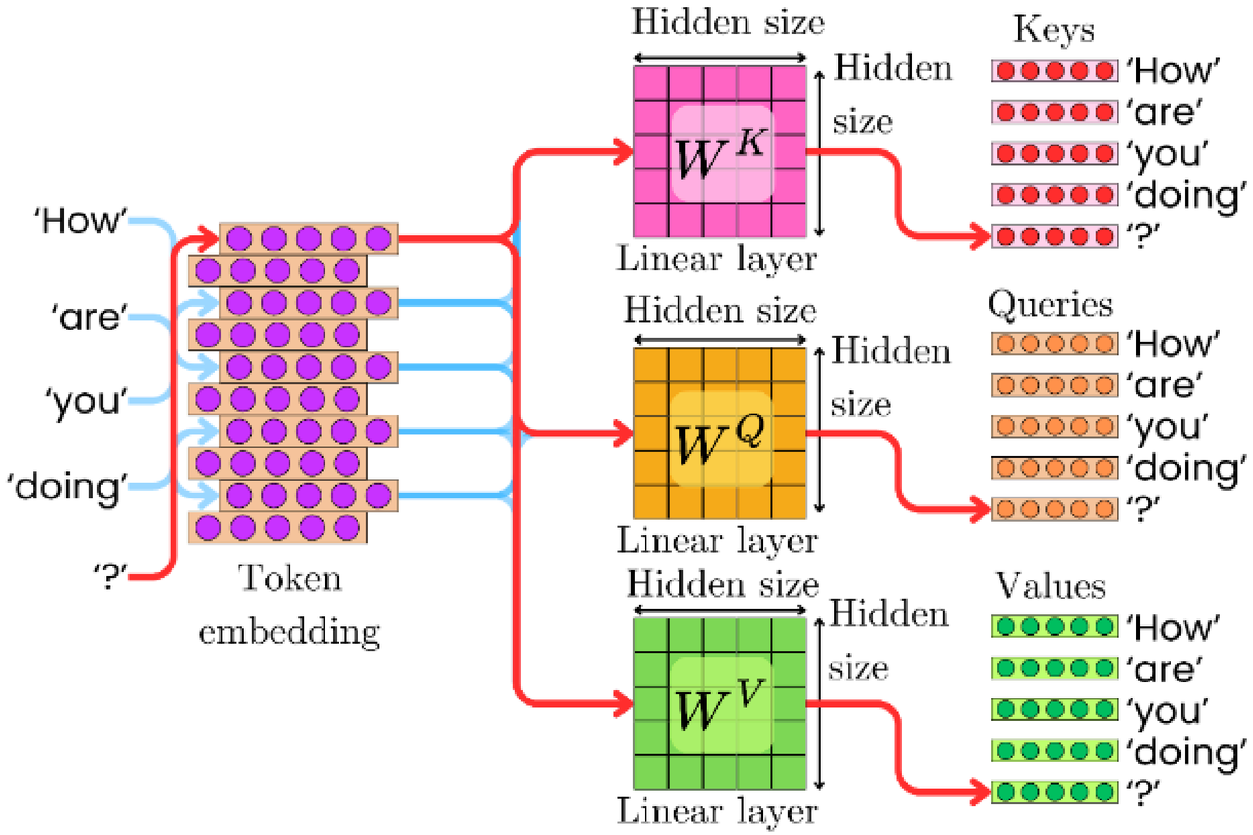


[Understanding The Self-Attention Mechanism]

Key, query, and value vectors



$$\mathbf{k}_i = W^K \mathbf{h}_i, \quad \text{keys}$$
$$\mathbf{q}_i = W^Q \mathbf{h}_i, \quad \text{queries}$$
$$\mathbf{v}_i = W^V \mathbf{h}_i, \quad \text{values}$$



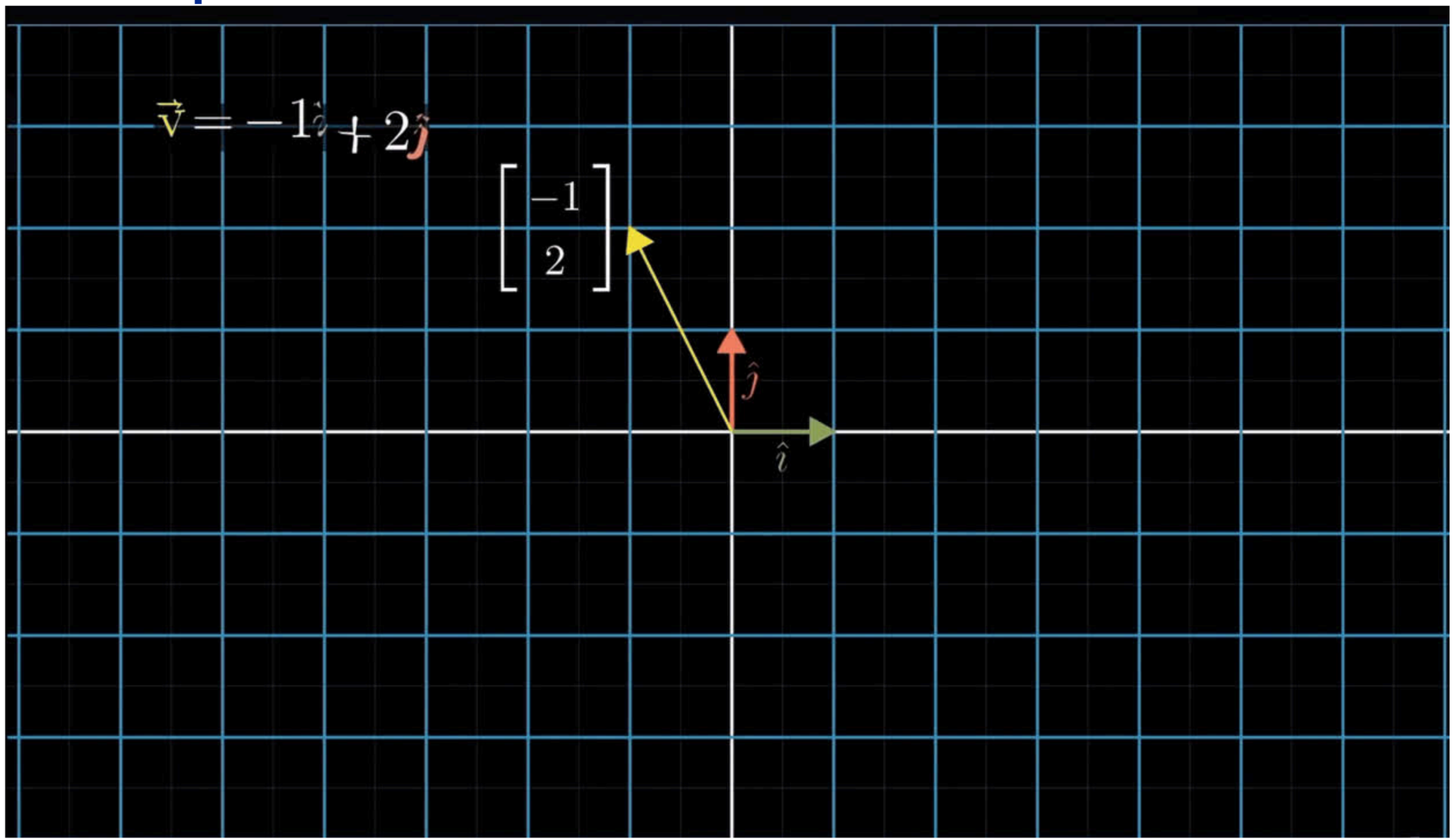
→ **Answers: What each token offers**

→ **Questions: What the current token wants**
→ **Position**

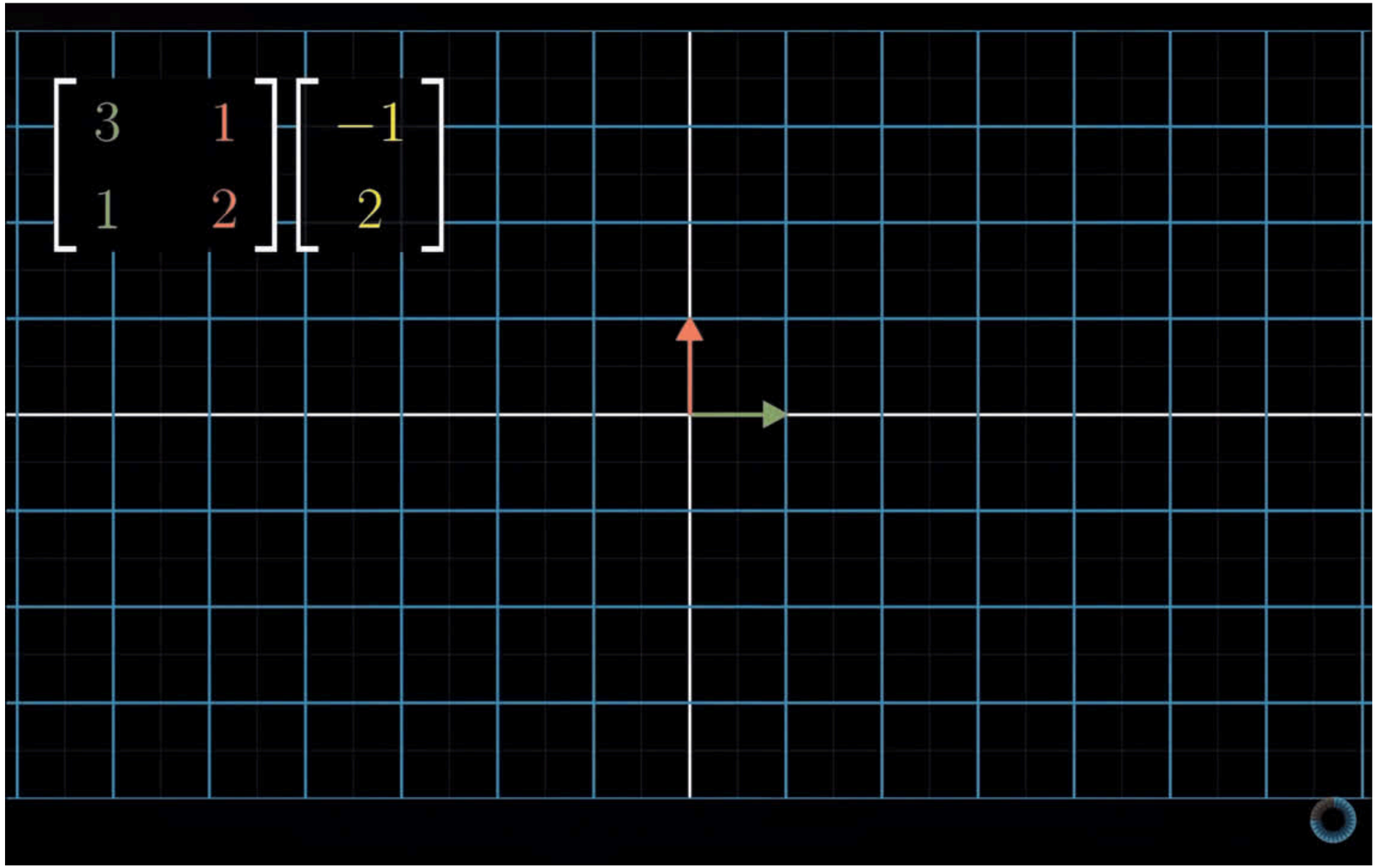
→ **Values: The content delivered if attention chooses that token**

How can Three weight matrices represent the meanings of Query, Key, and Value?

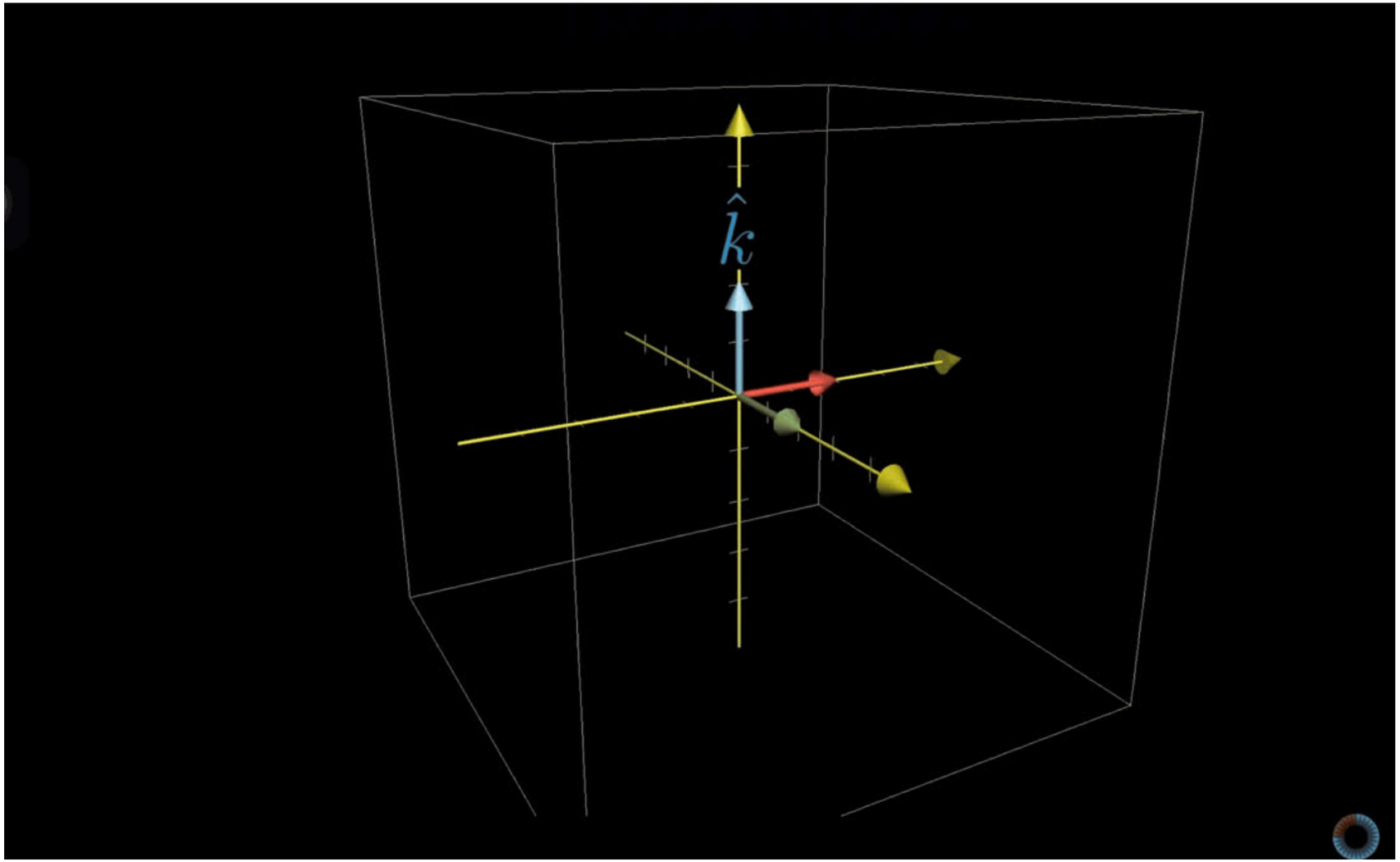
Recap: Matrix Multiplication



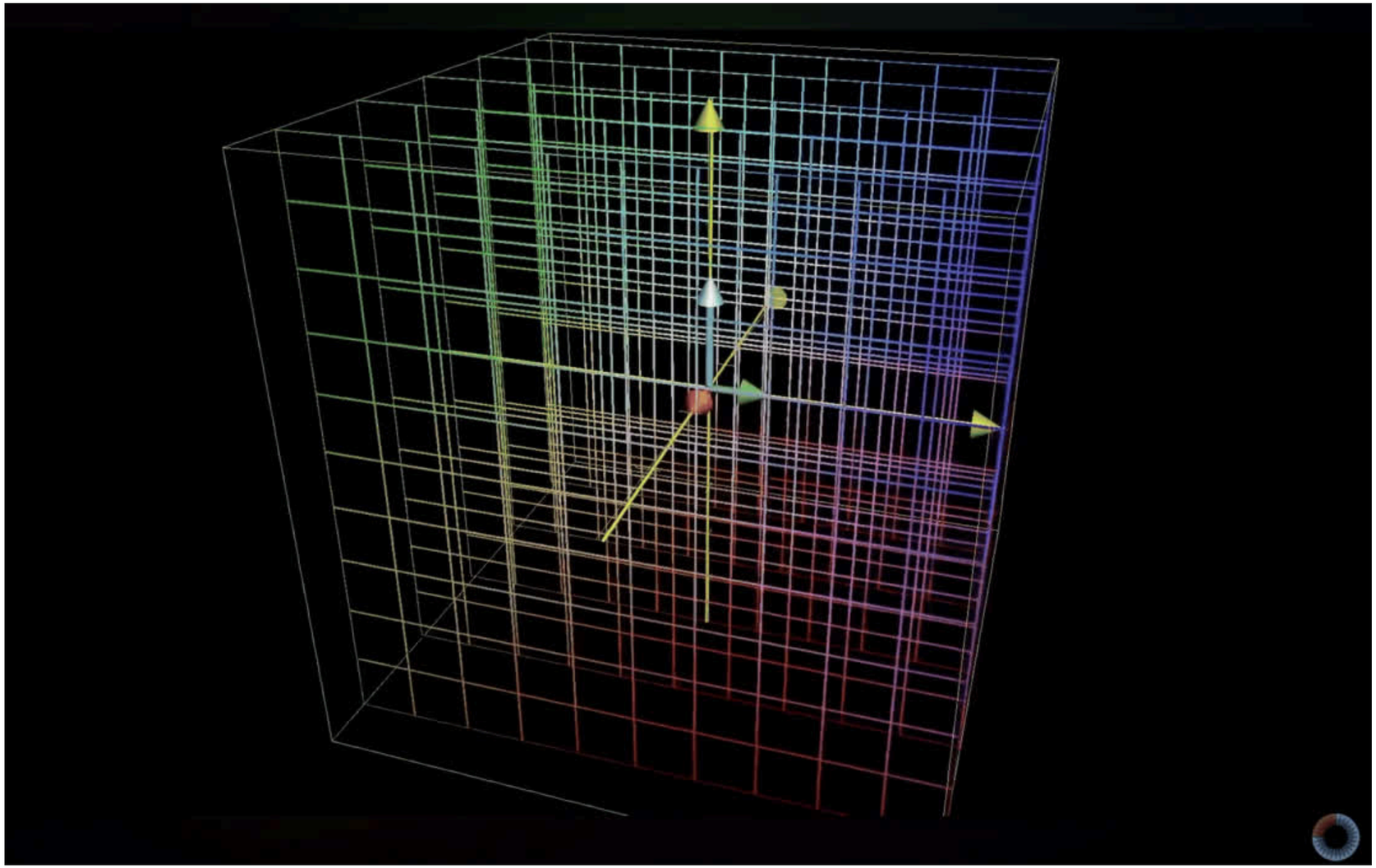
2D Transformation



3D Transformation

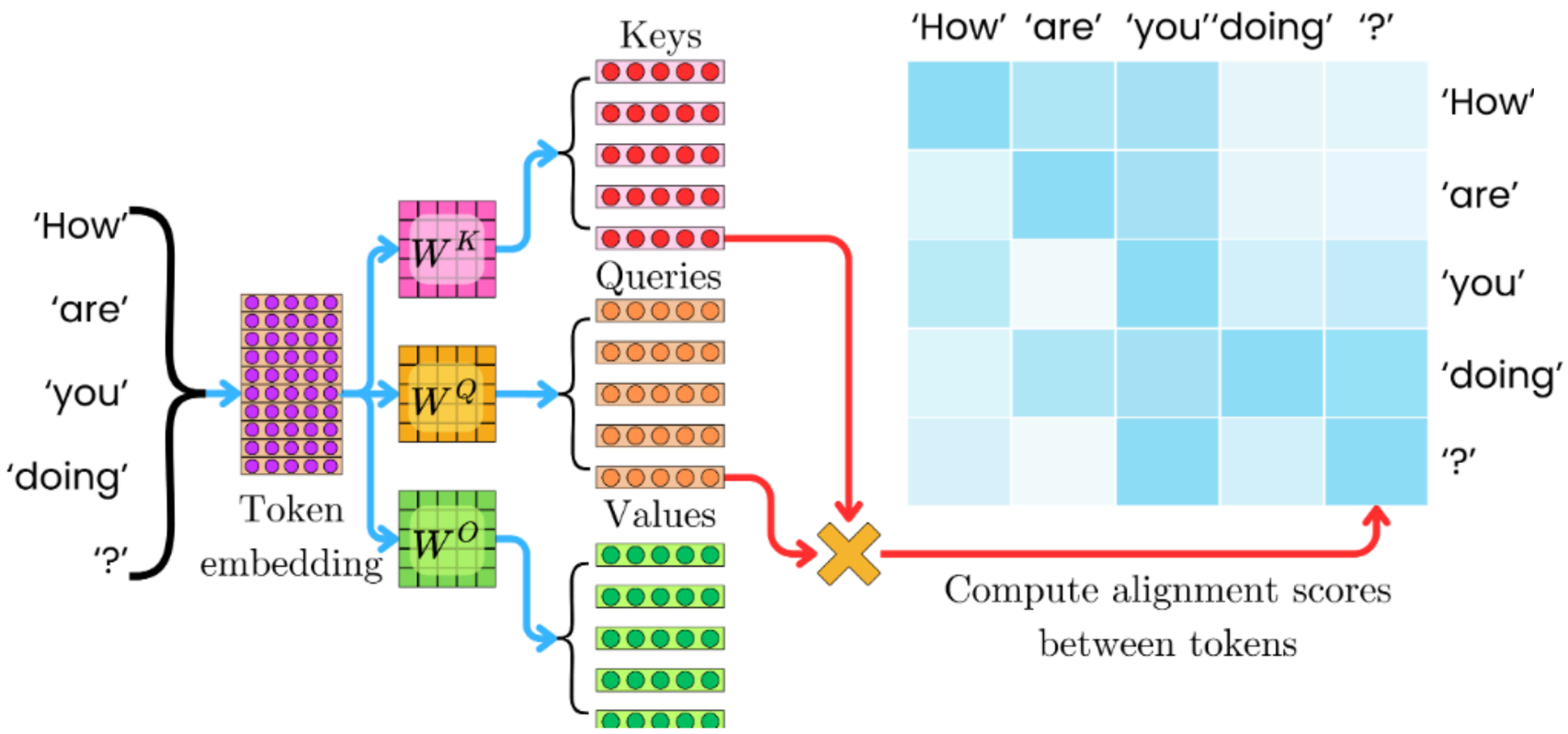


3D Transformation



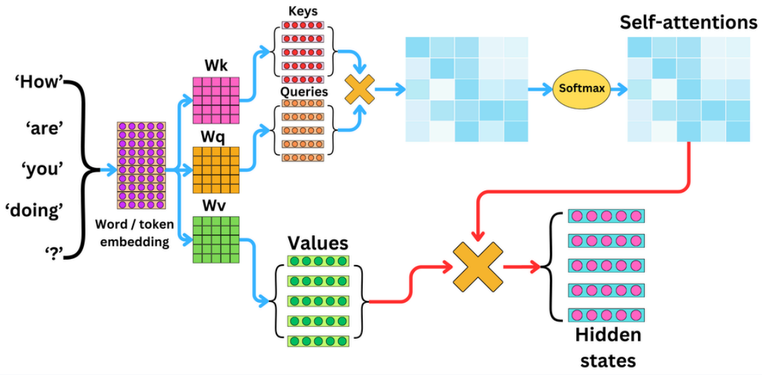
How are seft-attention weights created?

$$\vec{a} \cdot \vec{b} = |\vec{a}|\,|\vec{b}| \cos\theta$$



$$E = \frac{Q^\top K}{\sqrt{d_{\text{model}}}}$$

$$E = \frac{1}{\sqrt{d_{\text{model}}}} \begin{bmatrix} \mathbf{q}_1^\top \mathbf{k}_1 & \mathbf{q}_1^\top \mathbf{k}_2 & \cdots & \mathbf{q}_1^\top \mathbf{k}_N \\ \mathbf{q}_2^\top \mathbf{k}_1 & \mathbf{q}_2^\top \mathbf{k}_2 & \cdots & \mathbf{q}_2^\top \mathbf{k}_N \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{q}_N^\top \mathbf{k}_1 & \mathbf{q}_N^\top \mathbf{k}_2 & \cdots & \mathbf{q}_N^\top \mathbf{k}_N \end{bmatrix}$$

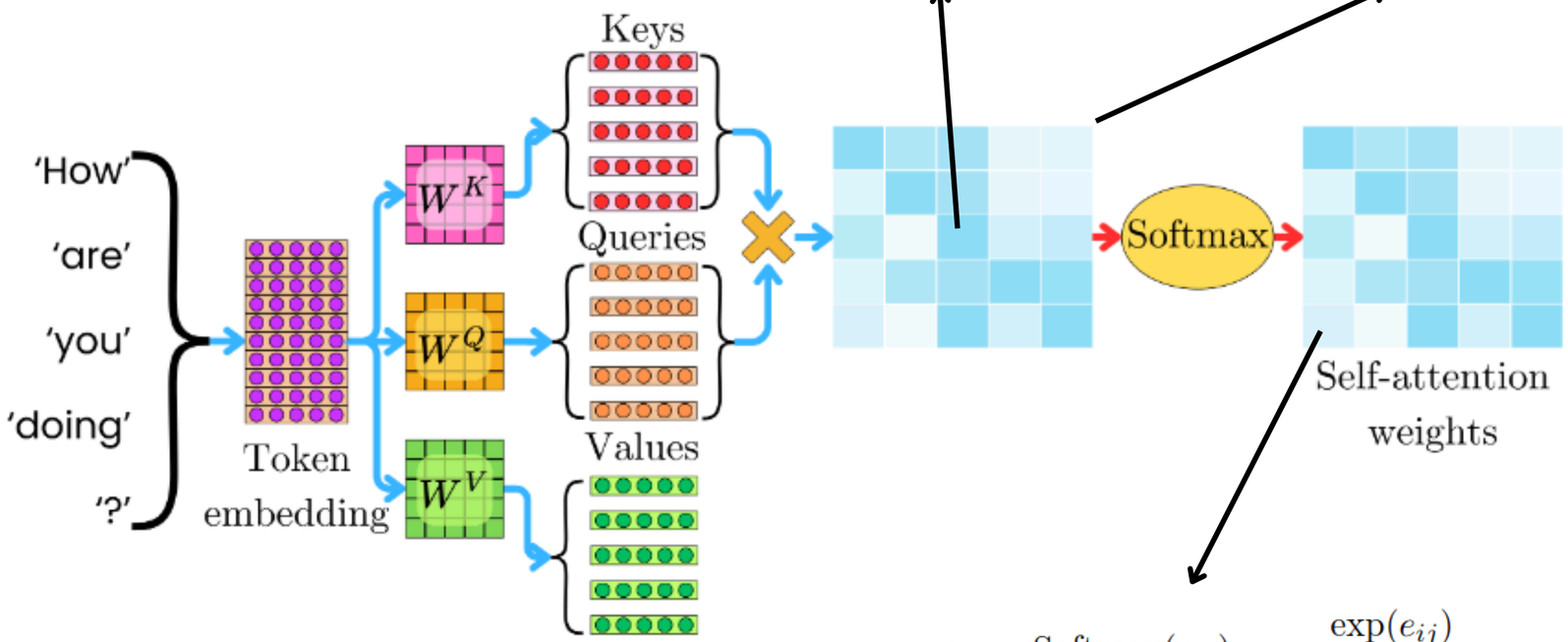


How are seft-attention weights created?

$$E = \frac{Q^\top K}{\sqrt{d_{\text{model}}}}$$

$$E = \frac{1}{\sqrt{d_{\text{model}}}} \begin{bmatrix} \mathbf{q}_1^\top \mathbf{k}_1 & \mathbf{q}_1^\top \mathbf{k}_2 & \cdots & \mathbf{q}_1^\top \mathbf{k}_N \\ \mathbf{q}_2^\top \mathbf{k}_1 & \mathbf{q}_2^\top \mathbf{k}_2 & \cdots & \mathbf{q}_2^\top \mathbf{k}_N \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{q}_N^\top \mathbf{k}_1 & \mathbf{q}_N^\top \mathbf{k}_2 & \cdots & \mathbf{q}_N^\top \mathbf{k}_N \end{bmatrix}$$

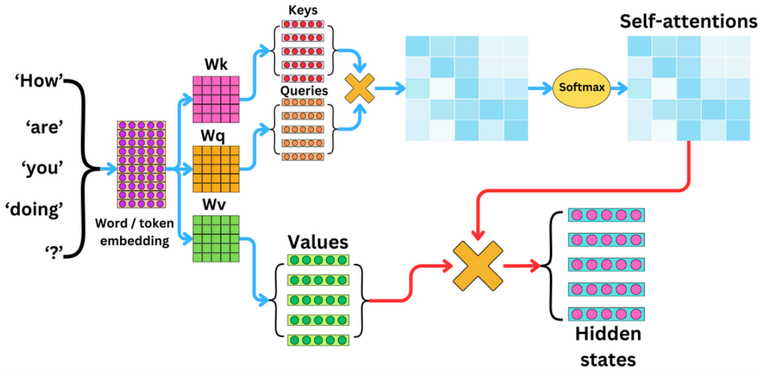
$$e_{ij} = \frac{\mathbf{k}_i^\top \mathbf{q}_j}{\sqrt{d_{\text{model}}}}$$



$$a_{ij} = \text{Softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{j=1}^{N} \exp(e_{ij})}$$

"a_{ij} represents how much the model considers token j when processing token i."

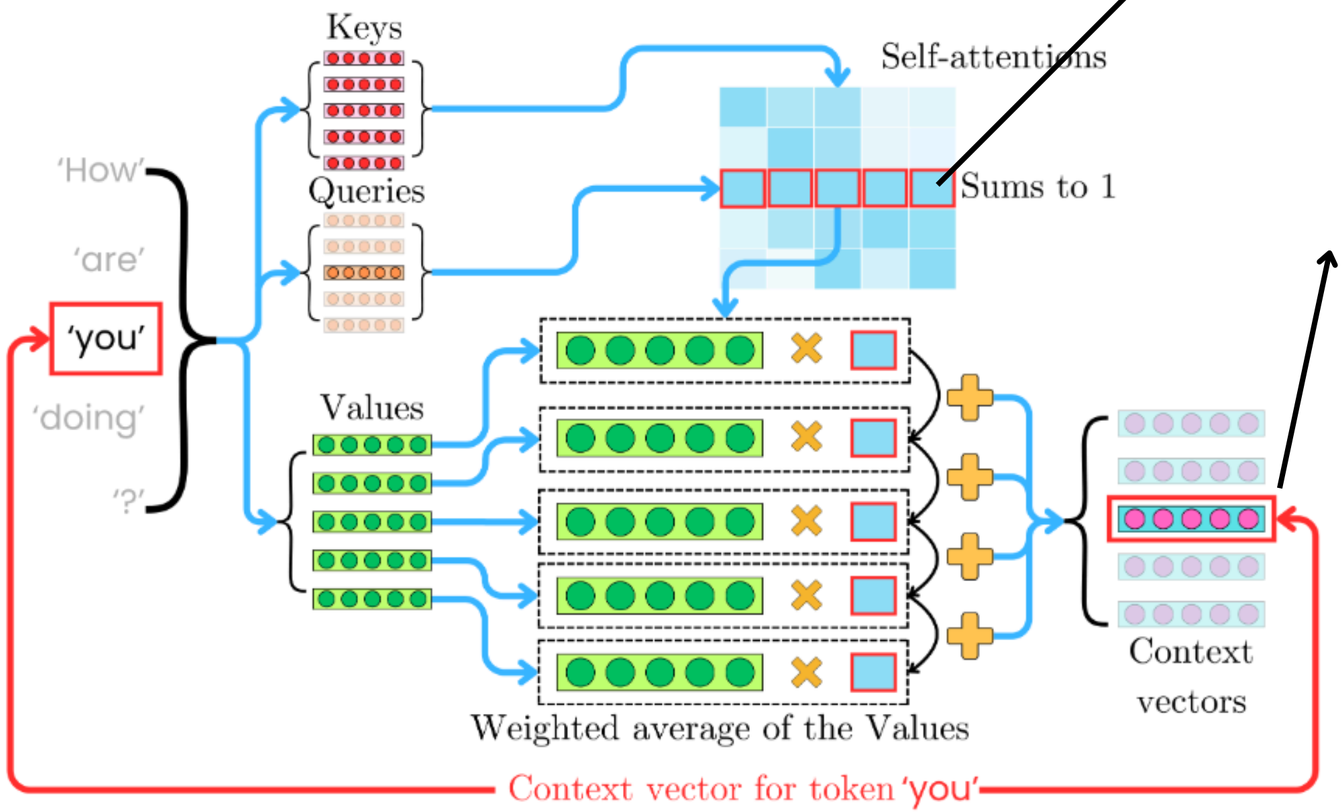
→ tell how much each token should pay attention to every other token.



How are context vectors created?

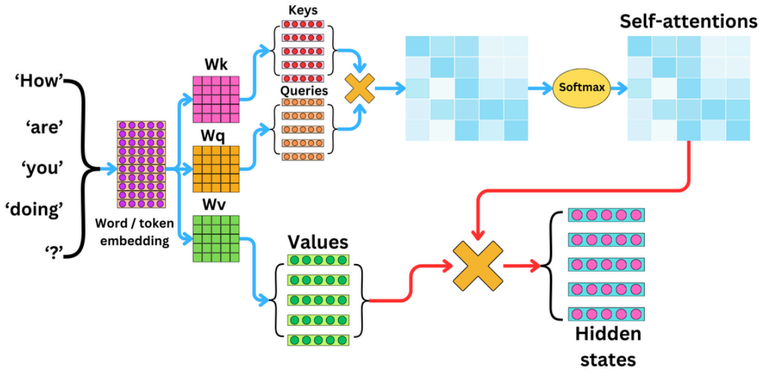
$$a_{ij} = \text{Softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{j=1}^{N} \exp(e_{ij})}$$

$$C = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{model}}}}\right)V$$

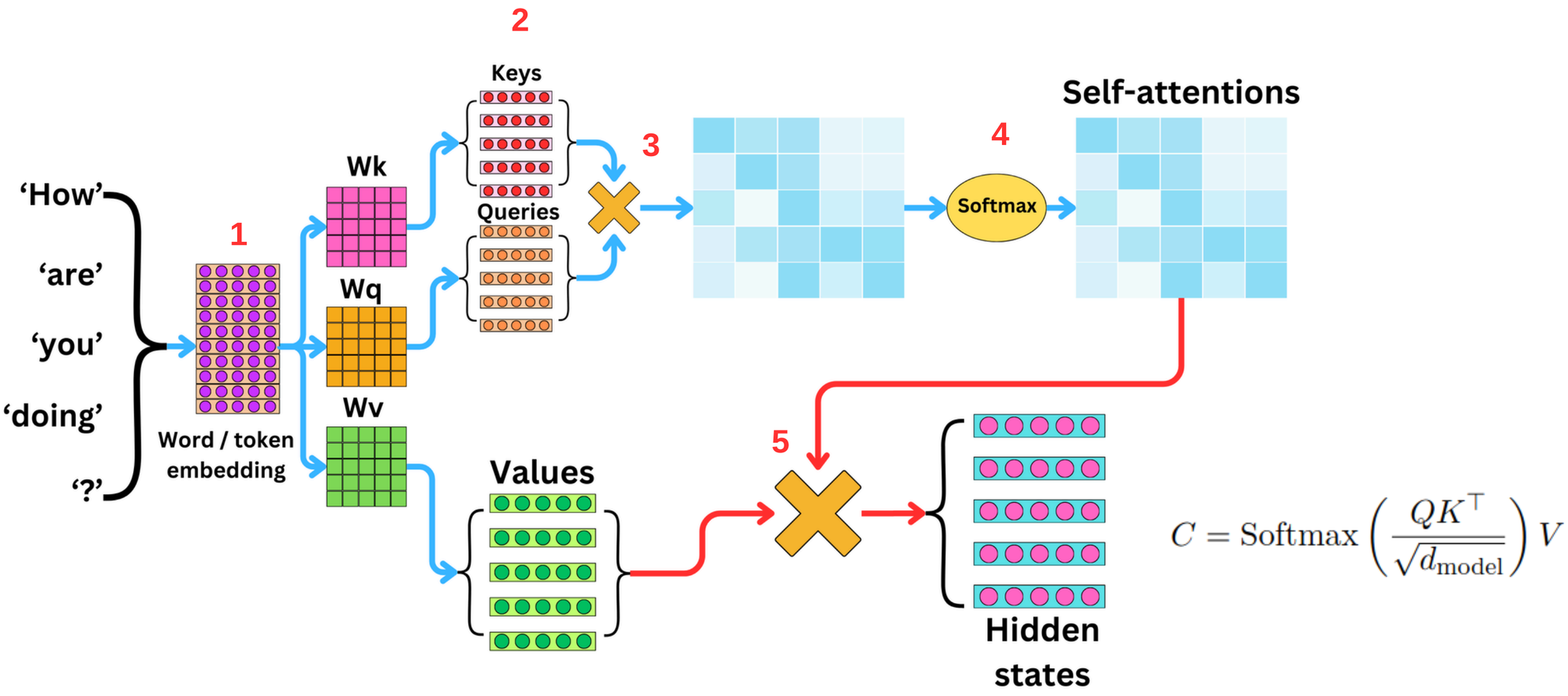


$$\mathbf{c}_i = \sum_{j=1}^{N} a_{ij}\mathbf{v}_j$$

→ present changes that every other token makes to the current token.



Self-Attention



$$C = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{model}}}}\right)V$$

Understanding The Self-Attention Mechanism

Multi-head Attention

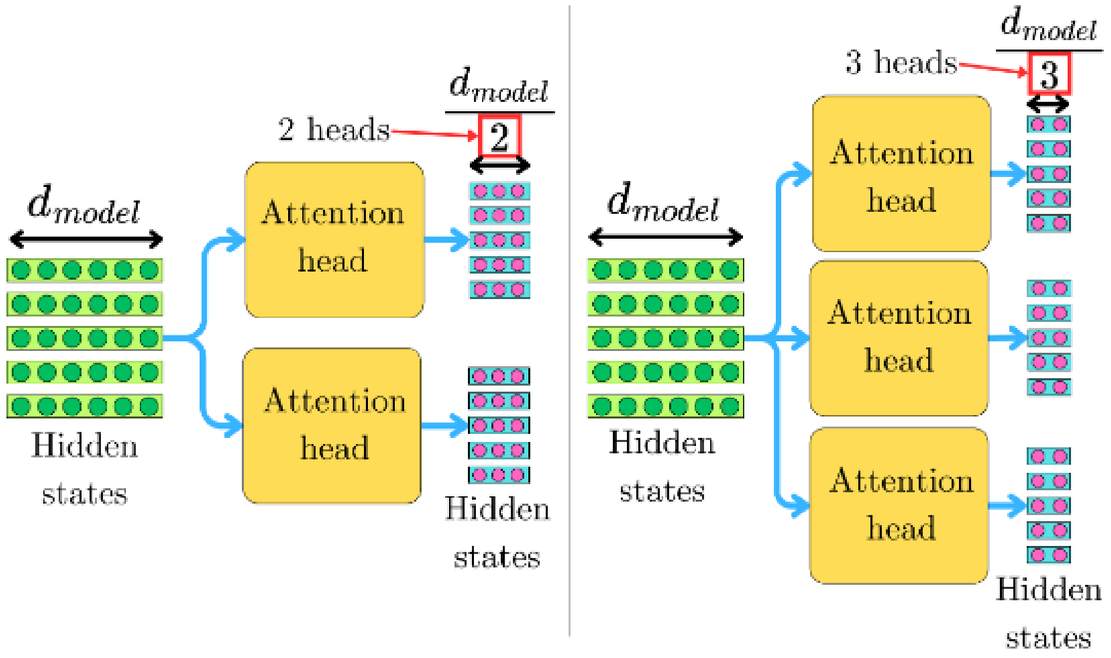
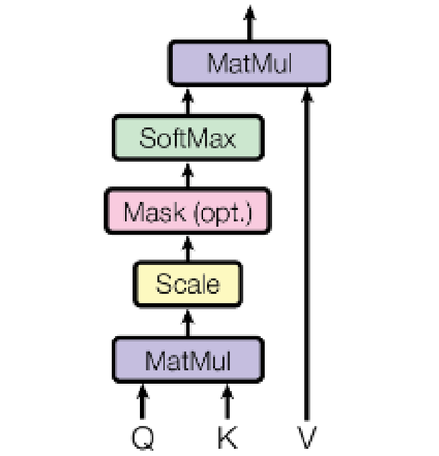
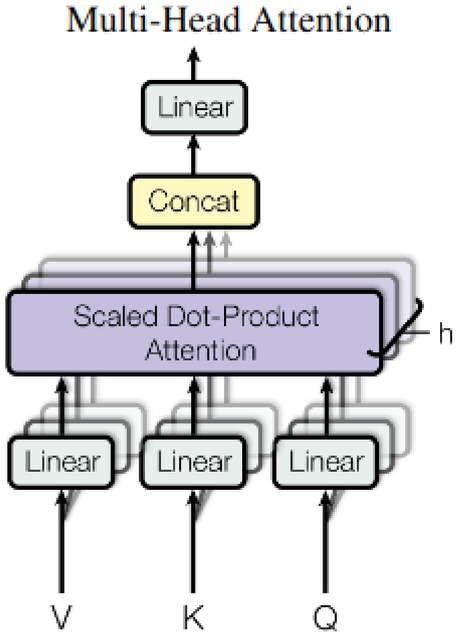
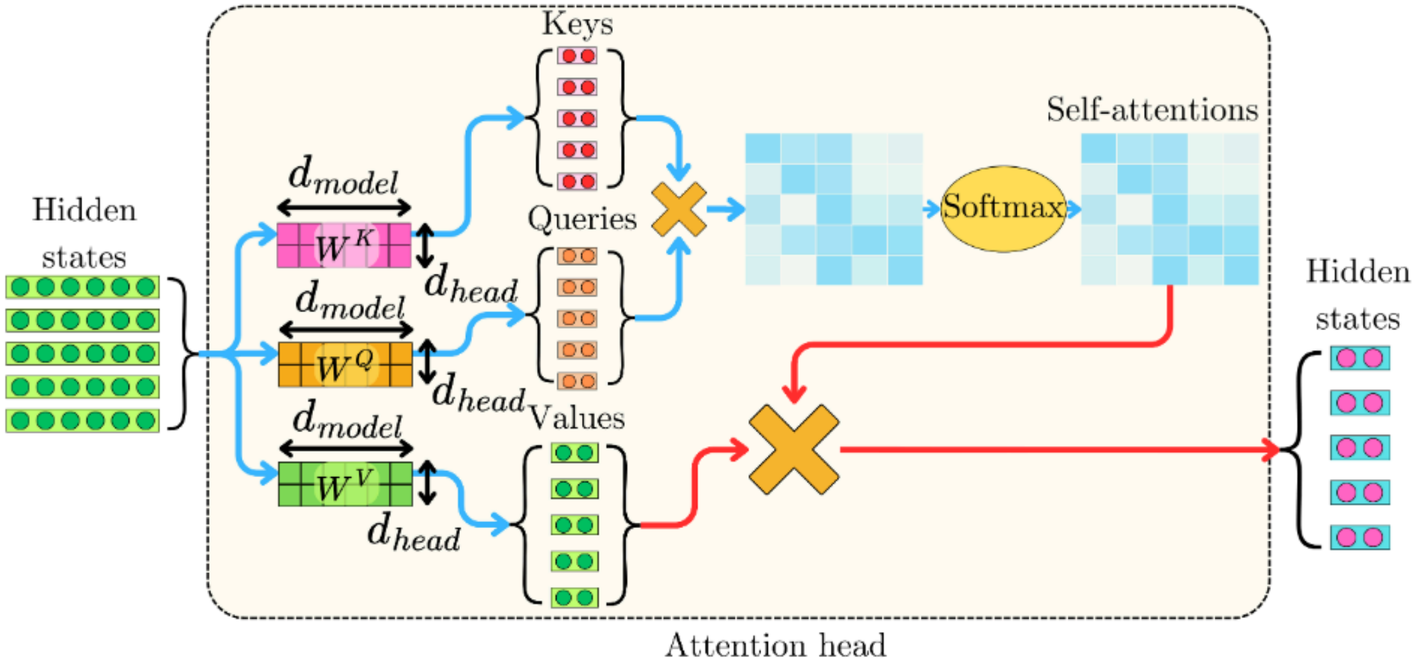


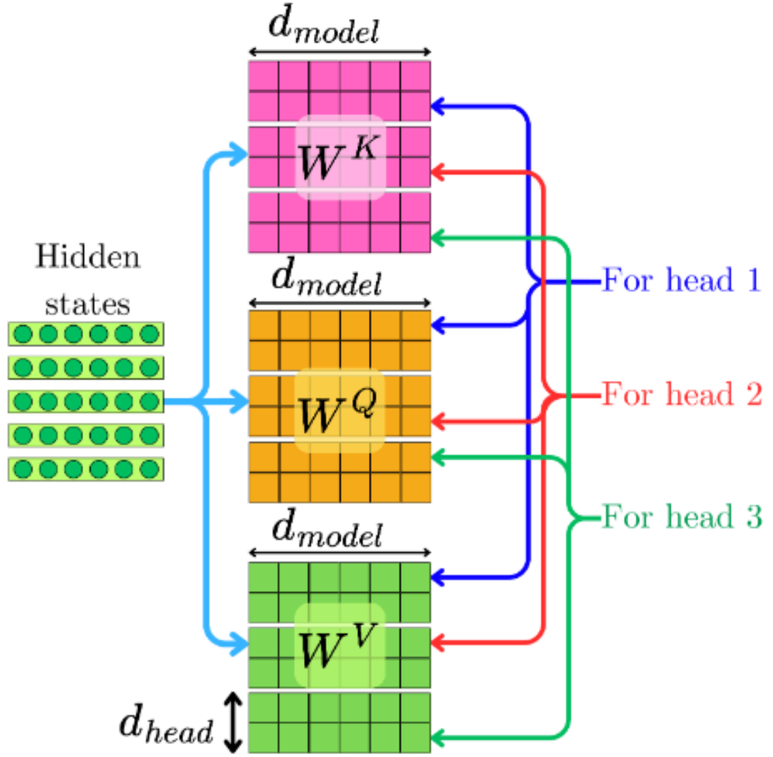
Fig. 1.7 *Each attention generates vectors of size* $d_{head} = \frac{d_{model}}{n_{head}}$*, depending of the number of heads.*





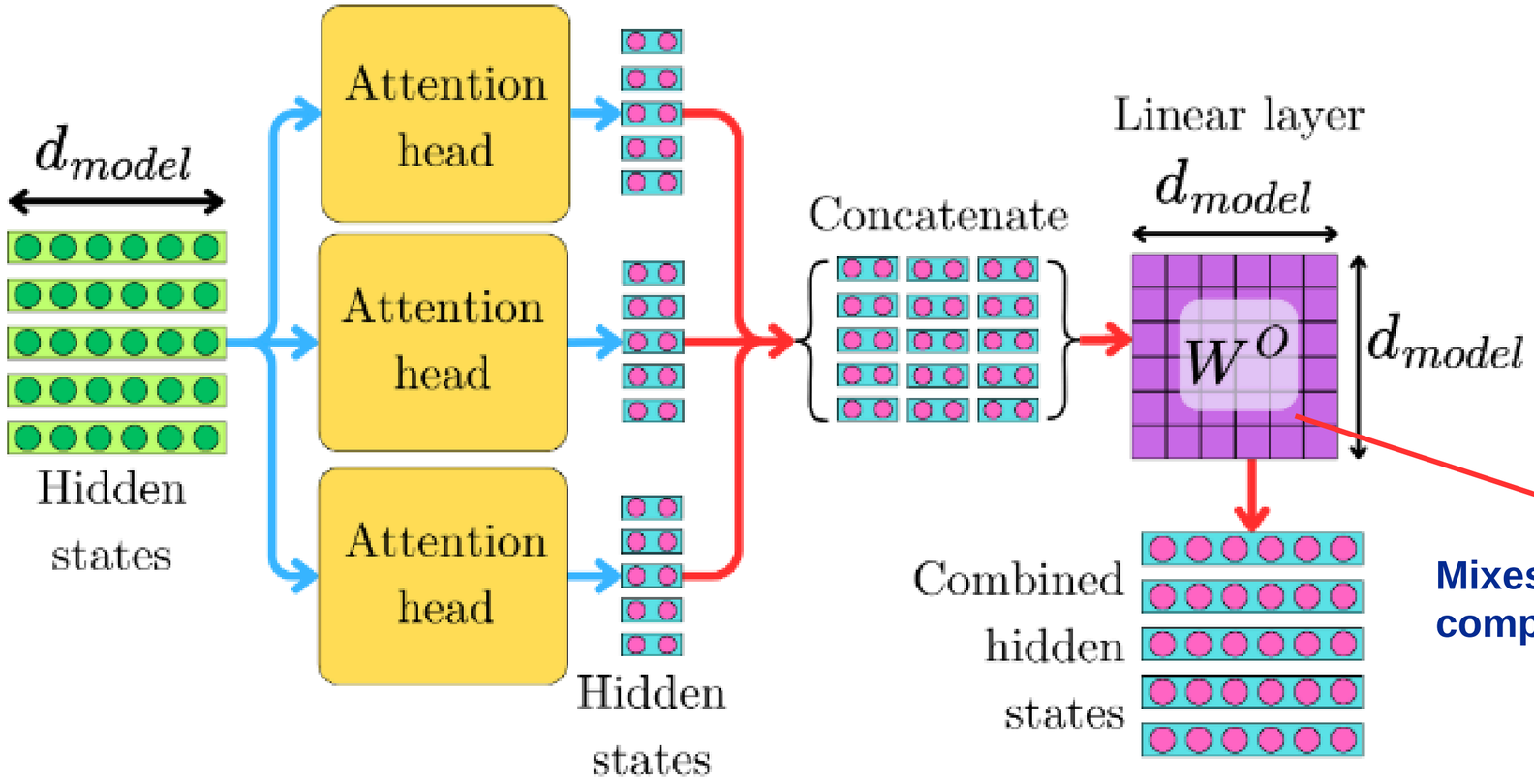
Multi-head Attention





Multi-head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



Mixes information from all heads and compresses it back into a single representation.

Efficient tensor operations for GPU computing

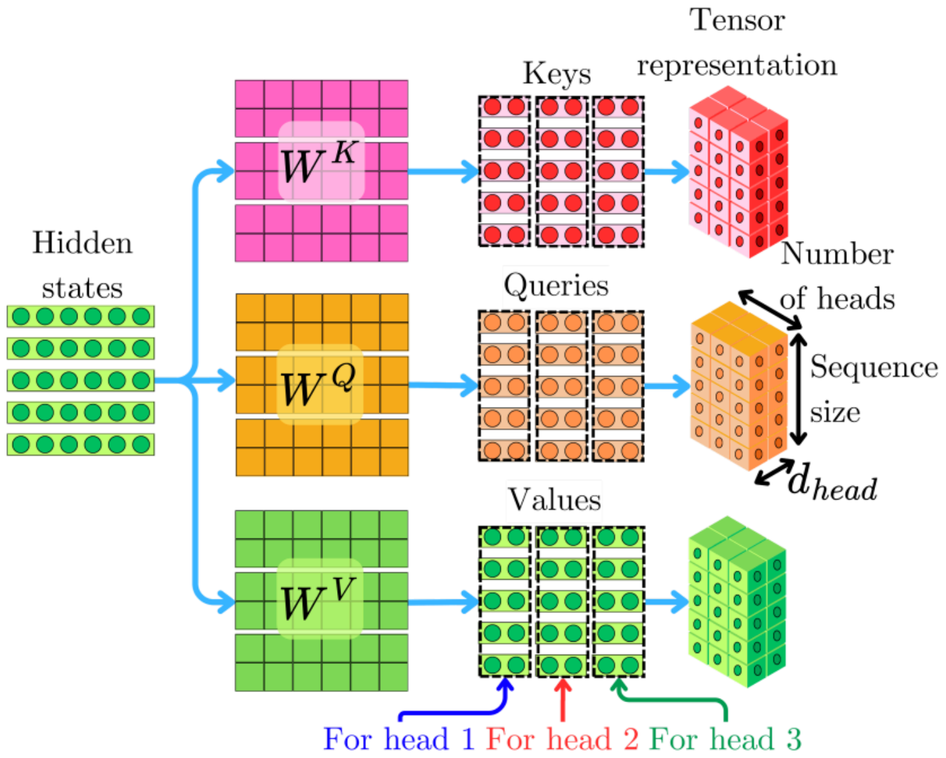


Fig. 1.11 *The keys, queries, and values are reshaped into 3-dimensional tensors with dimension $n_{head} \times d_{head} \times N$, where each slice of the tensors corresponds to one head.*

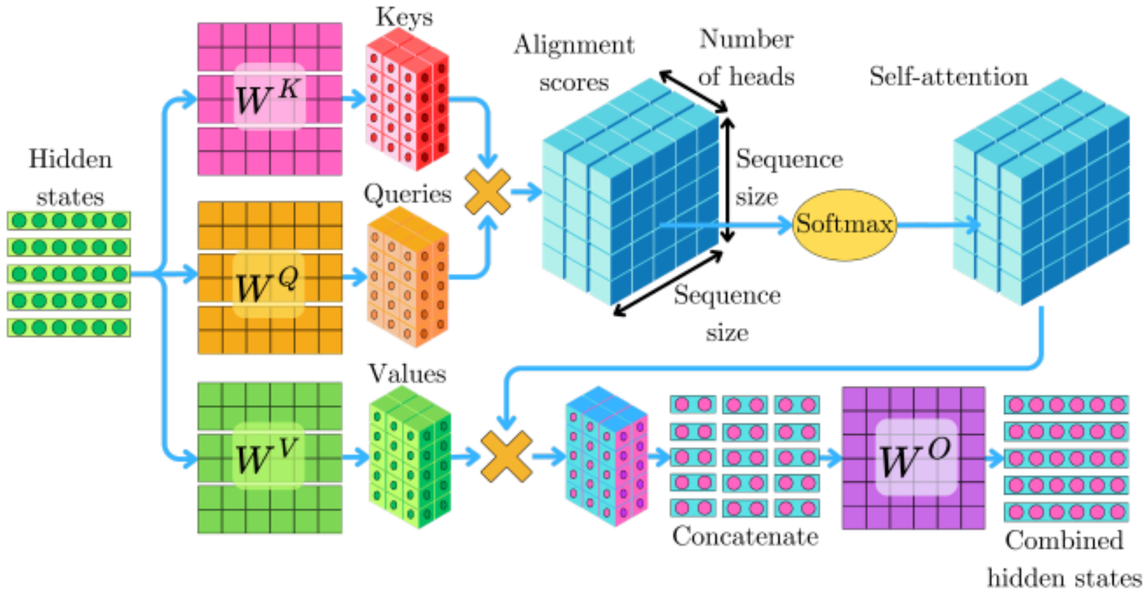


Fig. 1.12 *The computation of the attention and context vectors across multiple heads happens in parallel by making use of efficient tensor operations for GPU computing.*

Encoder

Prediction Head

Decoder

Feed Forward Layer

Layer Normalization

Multi-Head Attention!
(list of parallel self-attentions)

Residual Connection

Multi-Head Attention!
(list of parallel cross-attentions)

Masked Multi-Head Attention!
(list of parallel masked-attentions)

Embeddings
(Token and Positional)

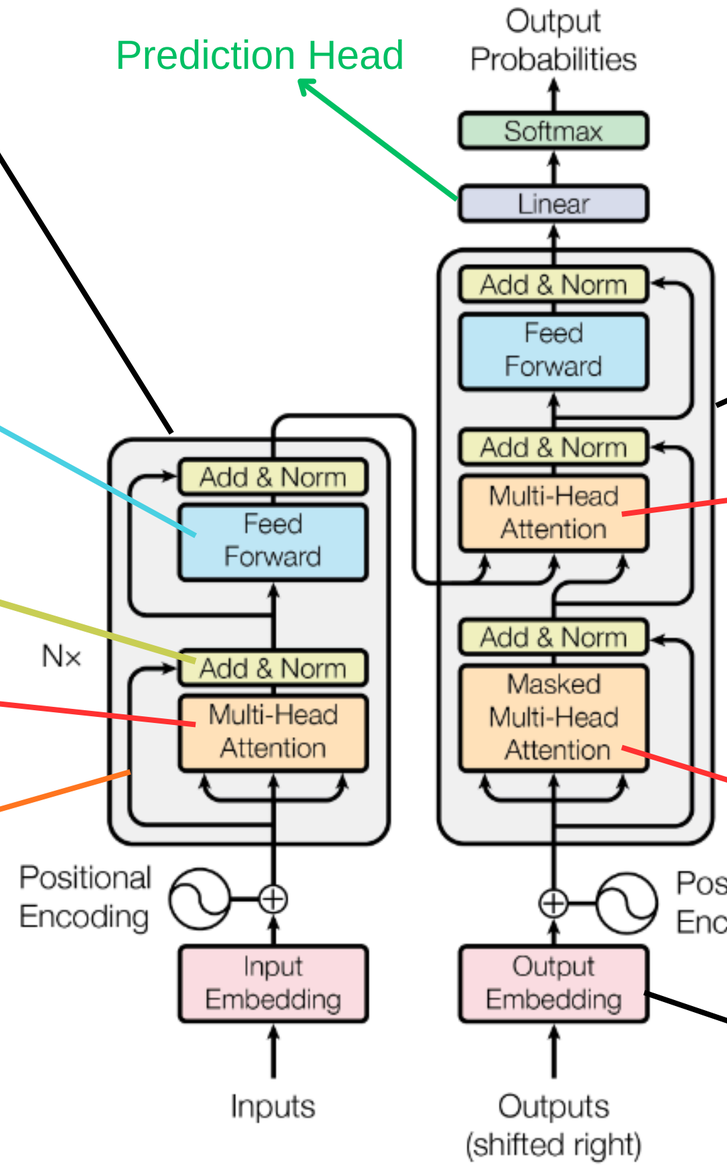


Figure 1: The Transformer - model architecture.

Nx: number of continously connected encoders/decoders

05

Residual Connection

Encoder

Prediction Head

Decoder

Feed Forward Layer

Layer Normalization

Multi-Head Attention!
(list of parallel self-attentions)

Residual Connection

Multi-Head Attention!
(list of parallel cross-attentions)

Masked Multi-Head Attention!
(list of parallel masked-attentions)

Embeddings
(Token and Positional)

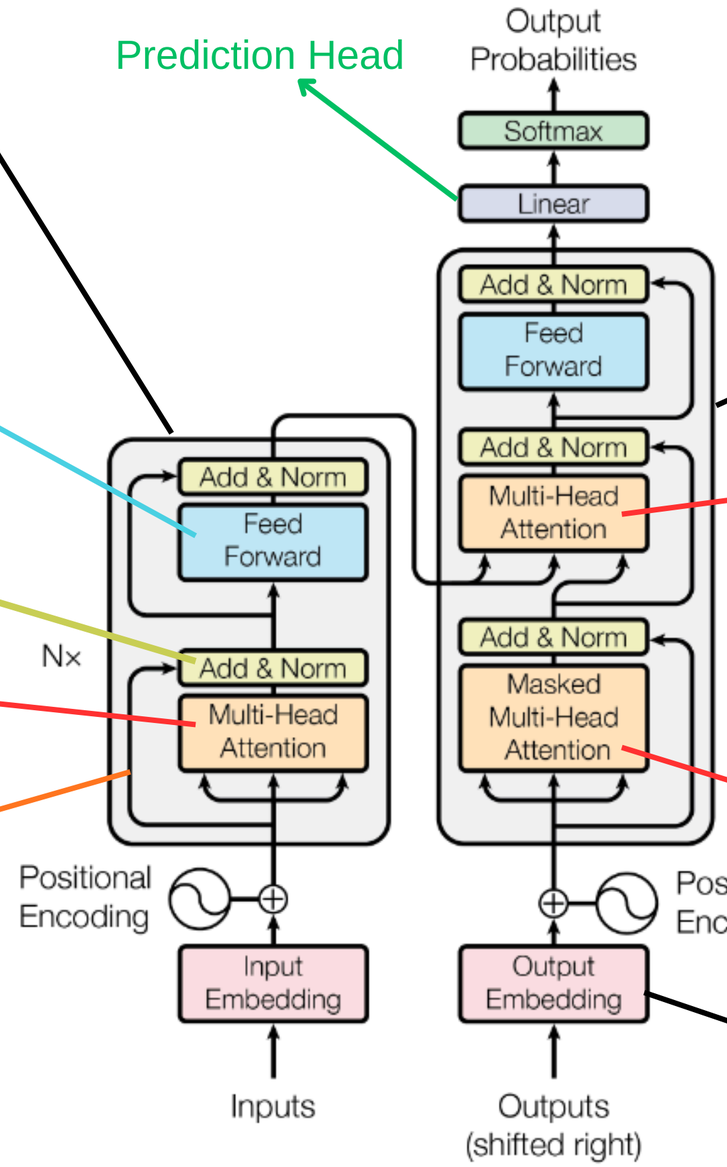
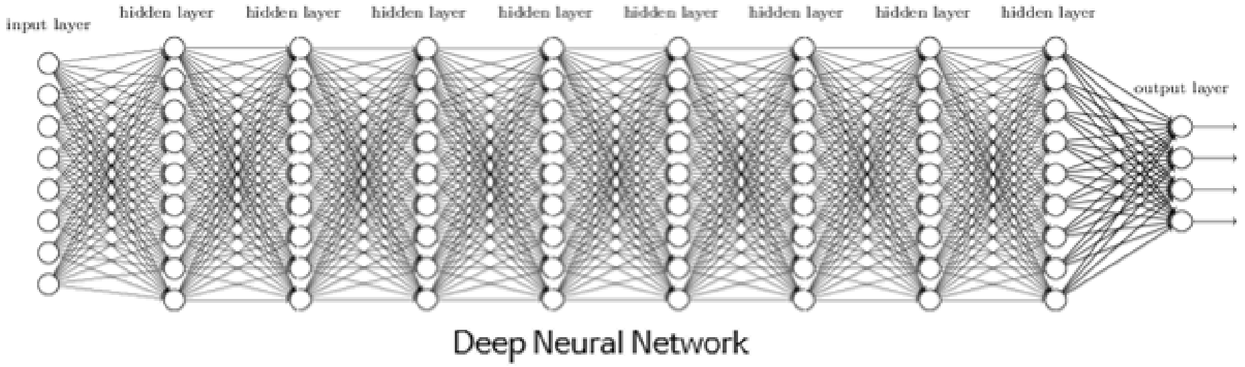


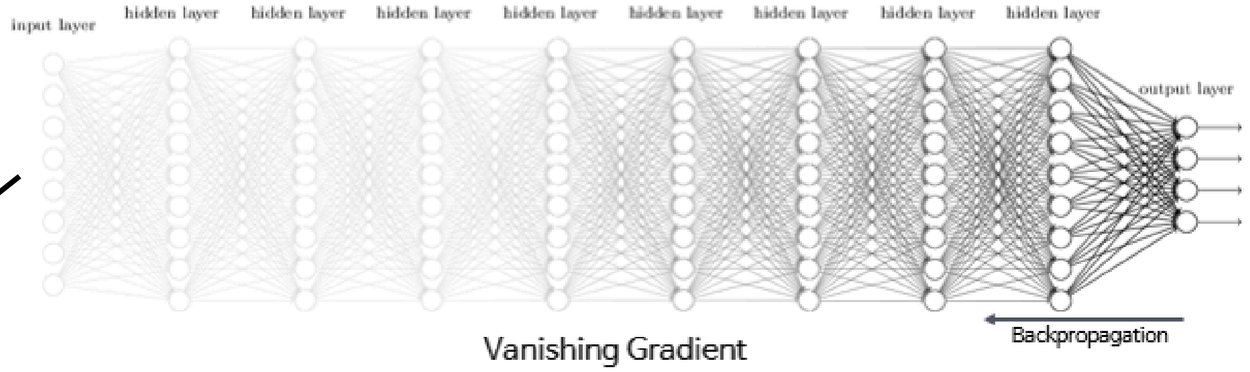
Figure 1: The Transformer - model architecture.

Nx: number of continously connected encoders/decoders

What does Residual Connection do?

- Handle vanishing gradient

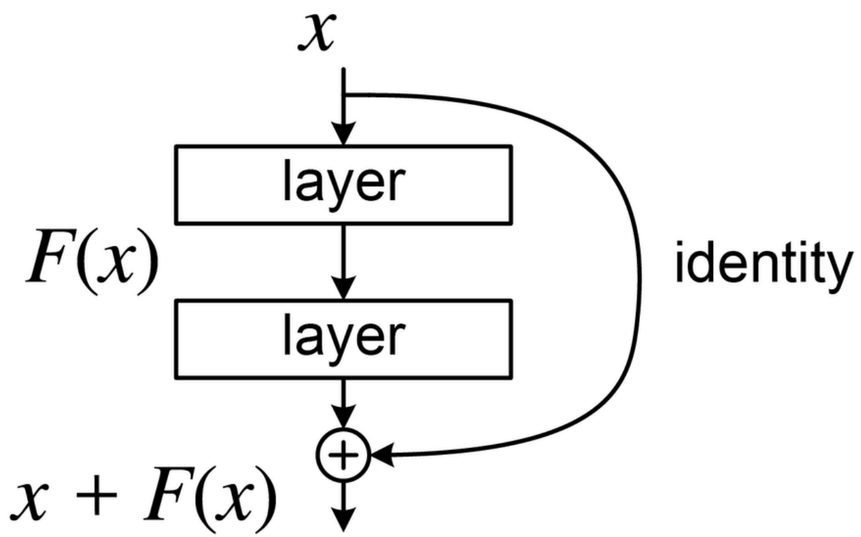




Gradient comes to zero

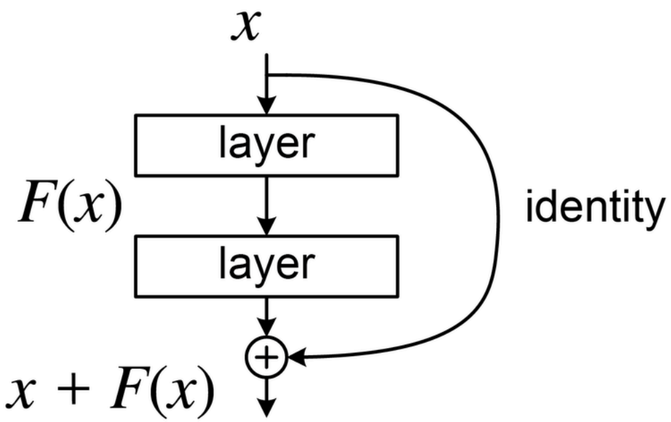
What does Residual Connection do?

- Residual Connection (or Skip Connection) lets the network skip some layers and add the original input directly to the output of those layers.



- x = input
- $F(x)$ = the transformation done by the block
- y = output after adding the shortcut

How can Skip Connection do this?



We have: $$y = F(x) + x$$

Compute gradient: $$\frac{\partial y}{\partial x} = 1 + \frac{\partial F}{\partial x}$$

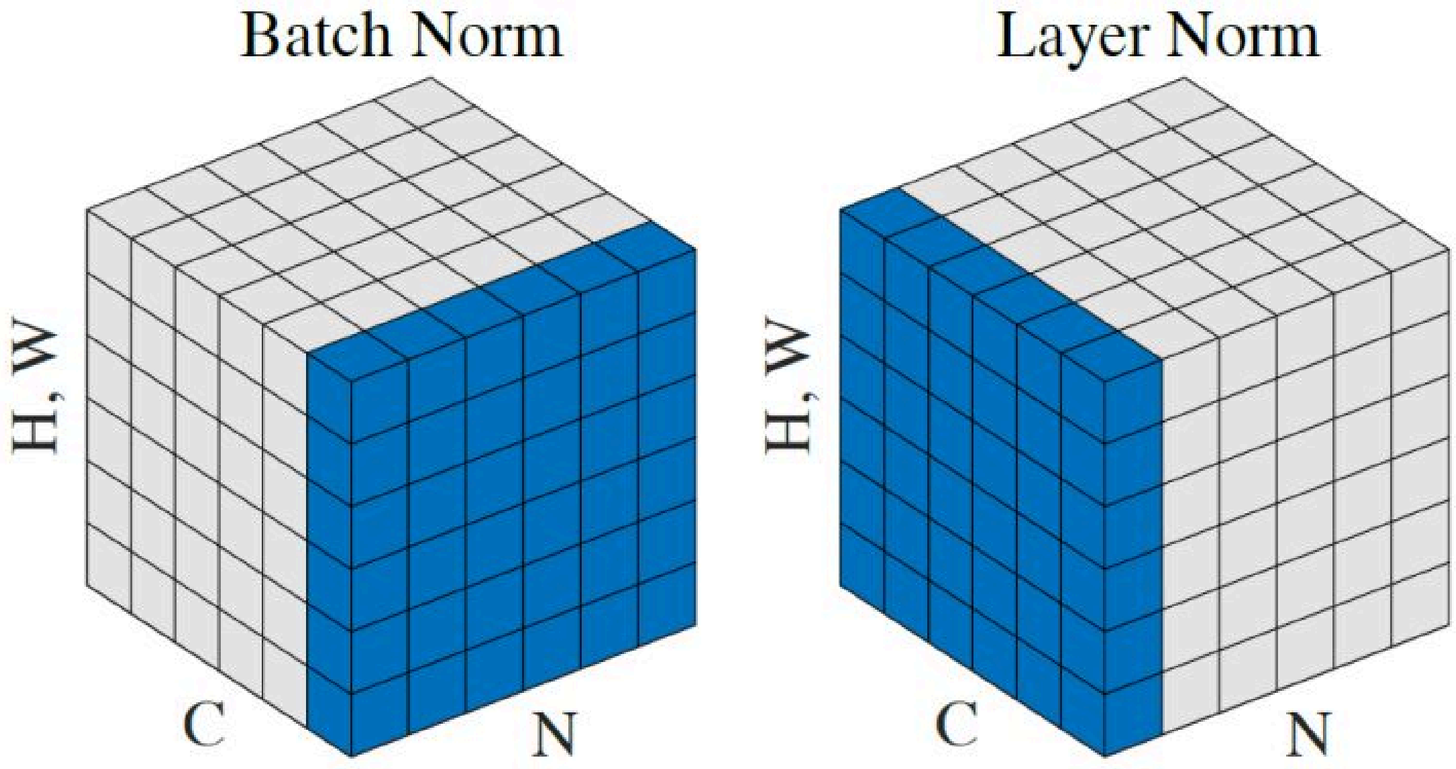
Even gradient of F(x) is small, gradients of y still pass through as “+1”.

Residuals make depth scalable!!

06

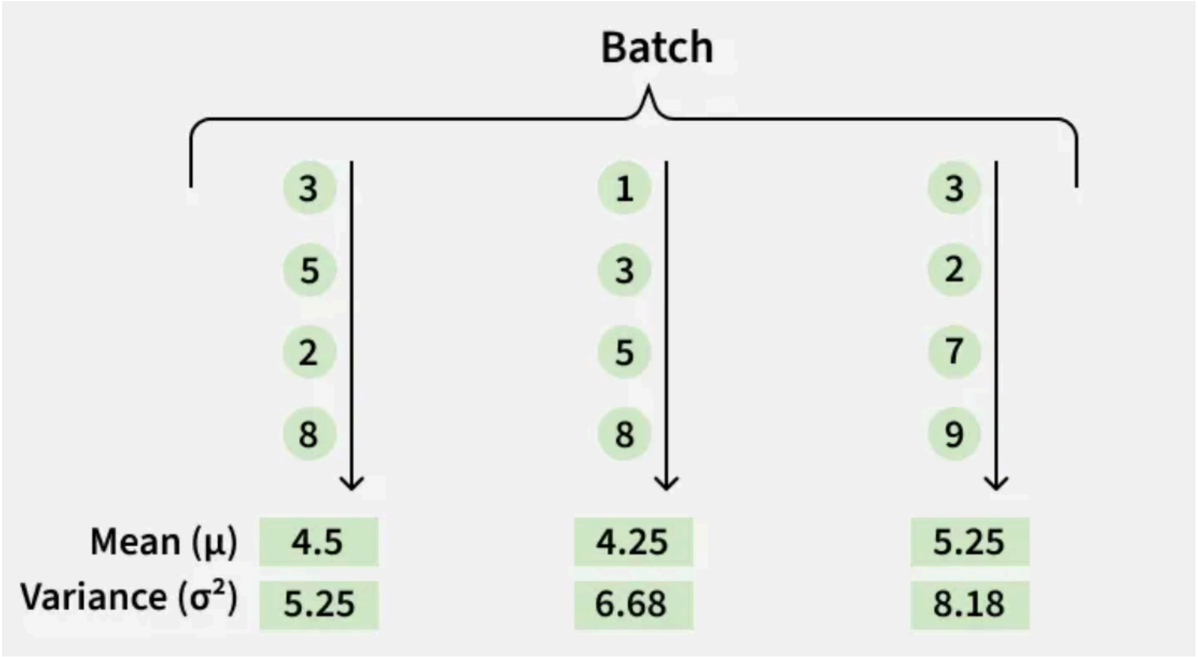
Layer Normalization

Batch Normalization vs Layer Normalization



Layer Normalization

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$



To represent a wide range of values, learnable parameters for scaling and shifting is applied

$$y_i = \gamma \hat{x}_i + \beta$$

07

Feed-Forward Network

Encoder

Prediction Head

Decoder

Feed Forward Layer

Layer Normalization

Multi-Head Attention!
(list of parallel self-attentions)

Residual Connection

Multi-Head Attention!
(list of parallel cross-attentions)

Masked Multi-Head Attention!
(list of parallel masked-attentions)

Embeddings
(Token and Positional)

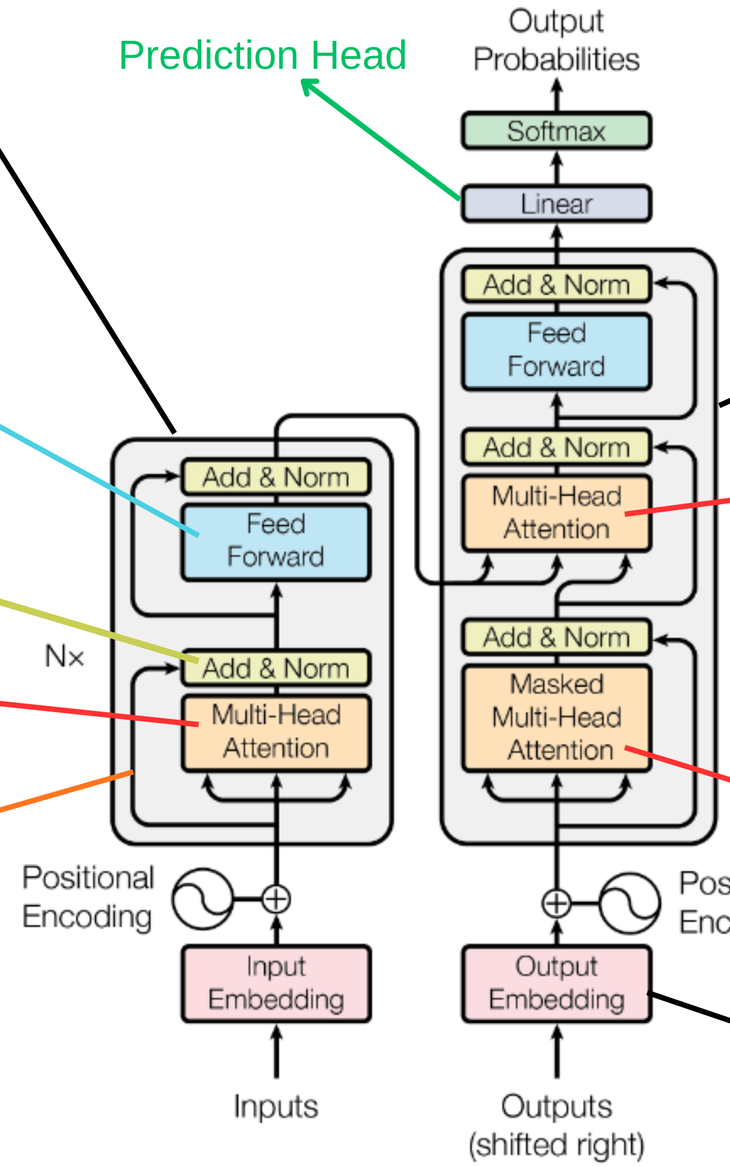


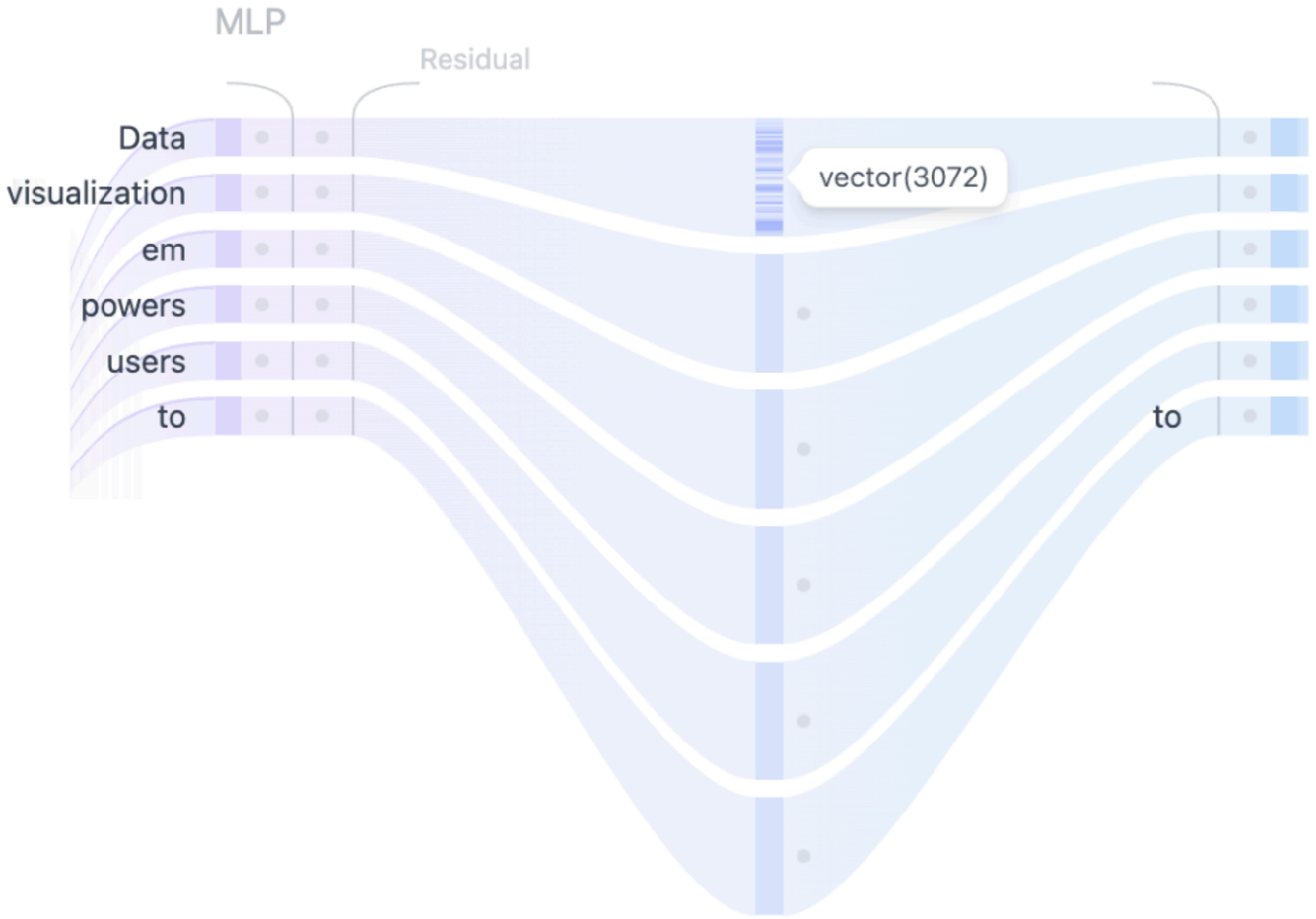
Figure 1: The Transformer - model architecture.

Nx: number of continously connected encoders/decoders

Step-by-step of FFN

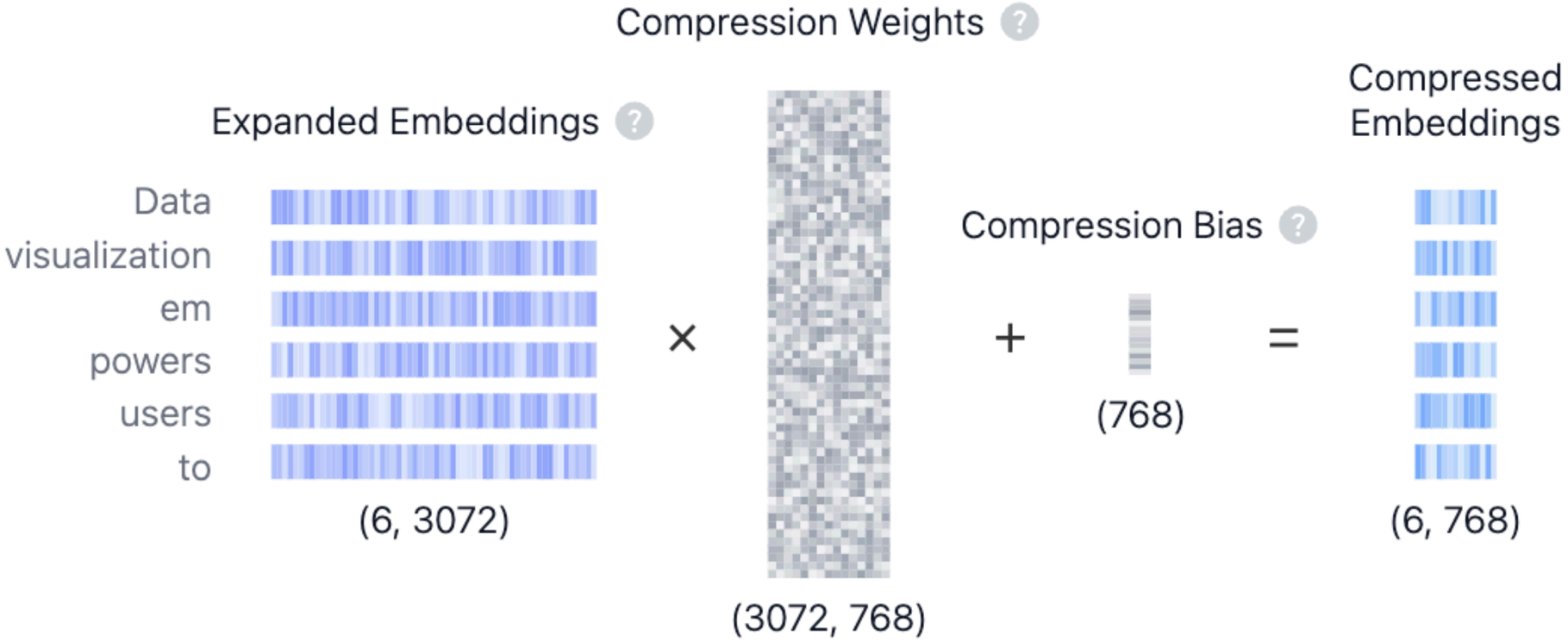
1. The input is expanded by a linear layer
2. Non-linearity is applied
3. The 2nd linear layer projects it back to its original dimensionality.

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$





$$\sum_{d=1}^{768} Emb_{id} \cdot Weights_{dj} + Bias_j = Expandedij$$



$$\sum_{d=1}^{768} Emb_{id} \cdot Weights_{dj} + Bias_j = Compressed_Embij$$

Why do we need FFN?

1. **Feature Enrichment:** Enhances token representations by adding depth to the model's understanding.
2. **Position Independence:** Processes tokens individually, complementing the global context learned from self-attention.
3. **Scalability:** Highly parallelizable, making it efficient for large datasets.

08

Mask and Cross Attention

Encoder

Prediction Head

Decoder

Feed Forward Layer

Layer Normalization

Multi-Head Attention!
(list of parallel self-attentions)

Residual Connection

Multi-Head Attention!
(list of parallel cross-attentions)

Masked Multi-Head Attention!
(list of parallel masked-attentions)

Embeddings
(Token and Positional)

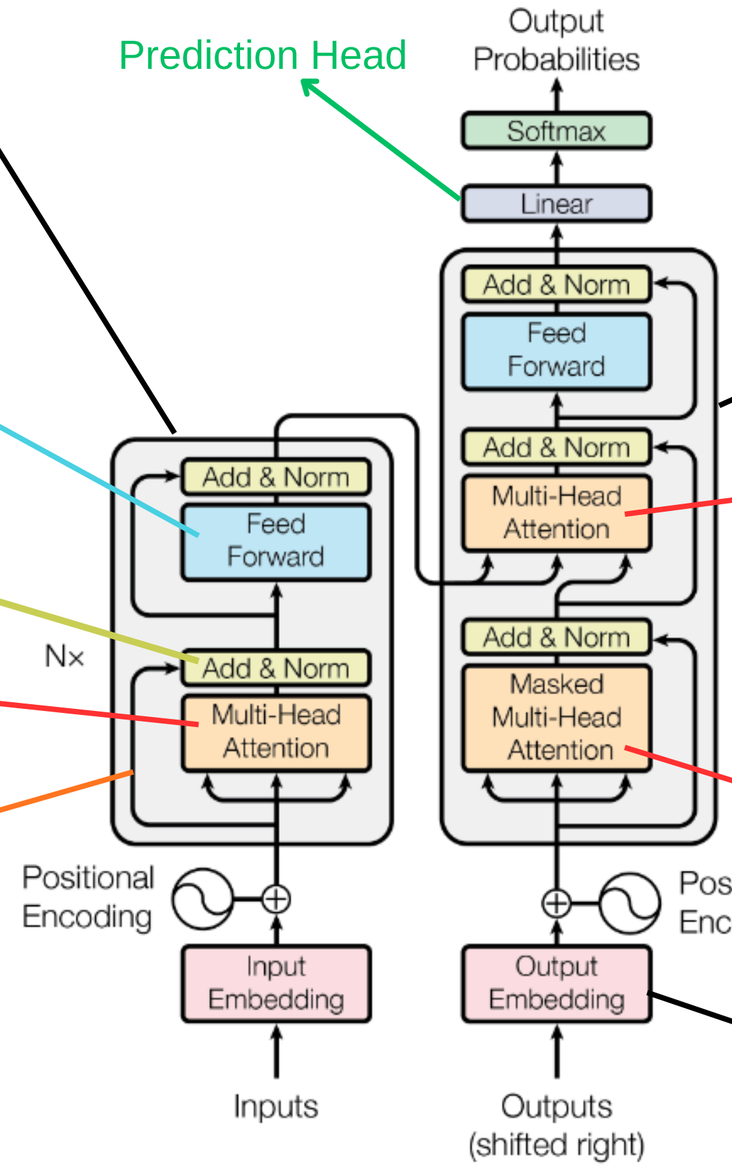
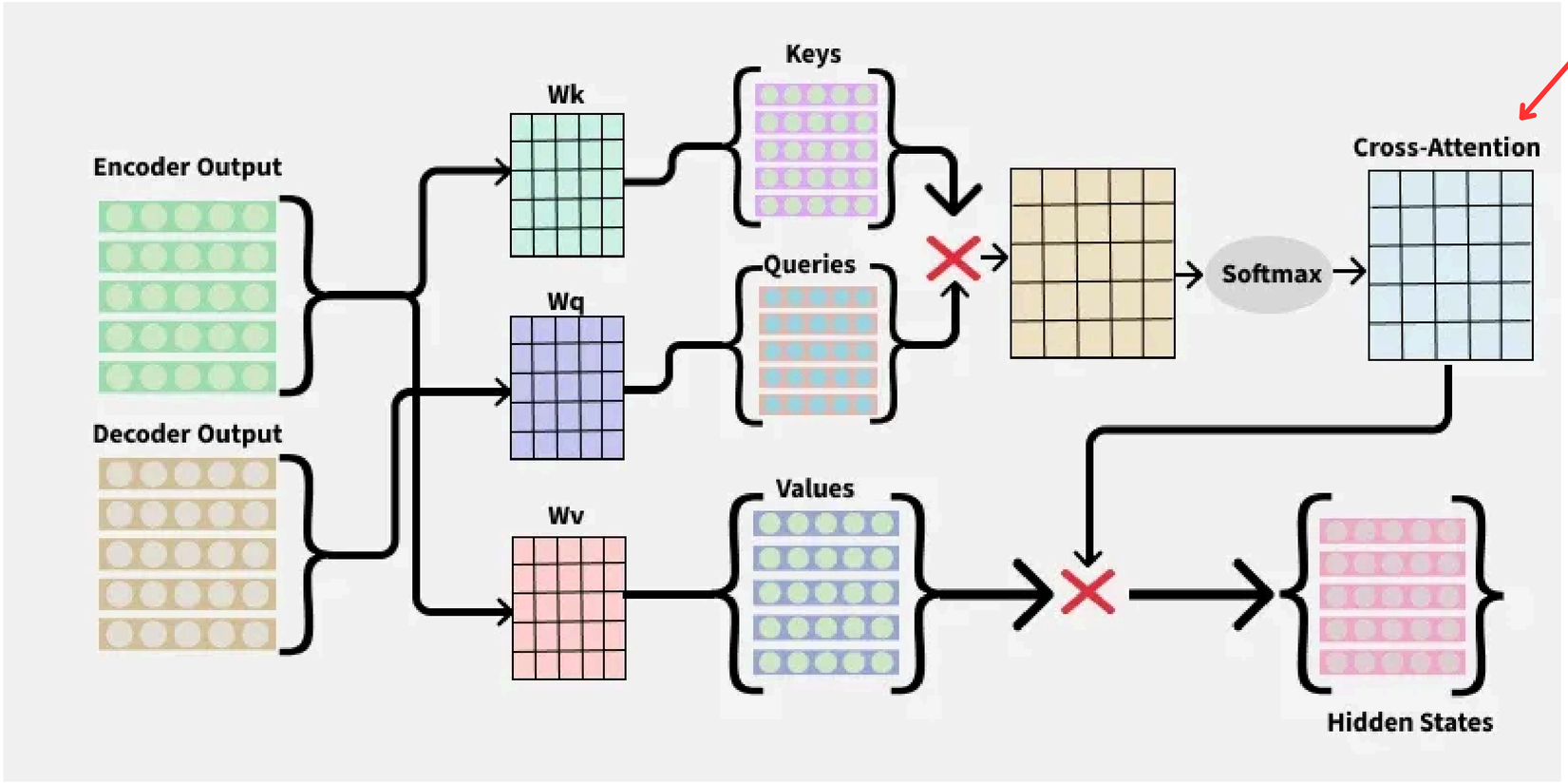


Figure 1: The Transformer - model architecture.

Nx: number of continously connected encoders/decoders

Cross Attention



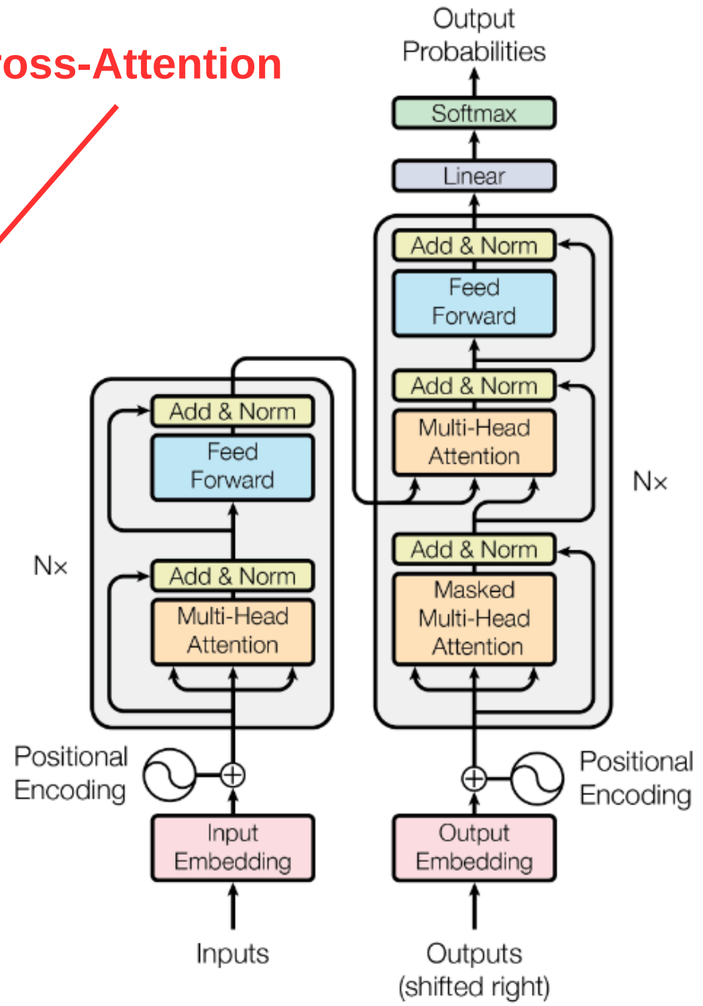
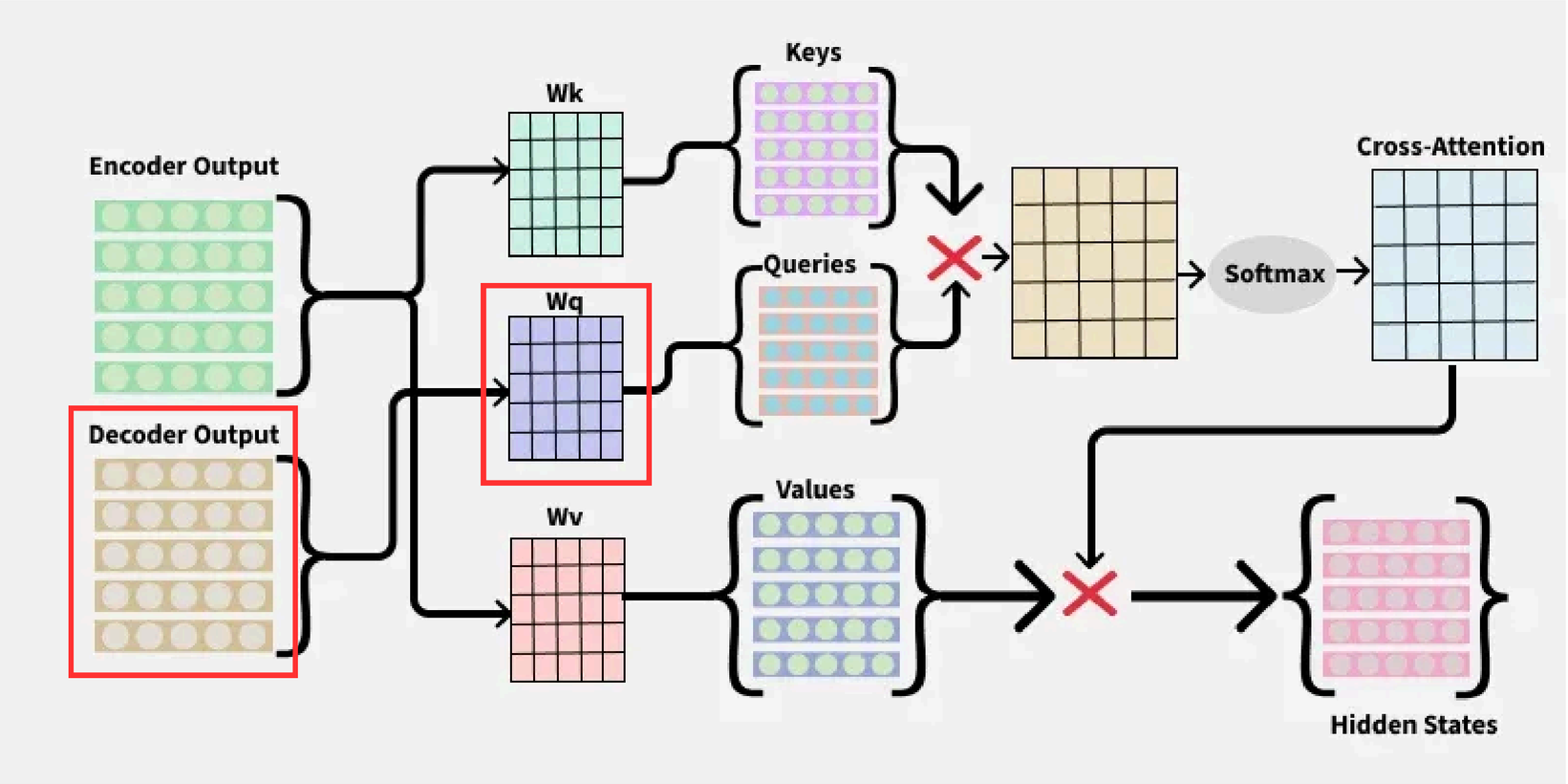


Figure 1: The Transformer - model architecture.

Cross Attention



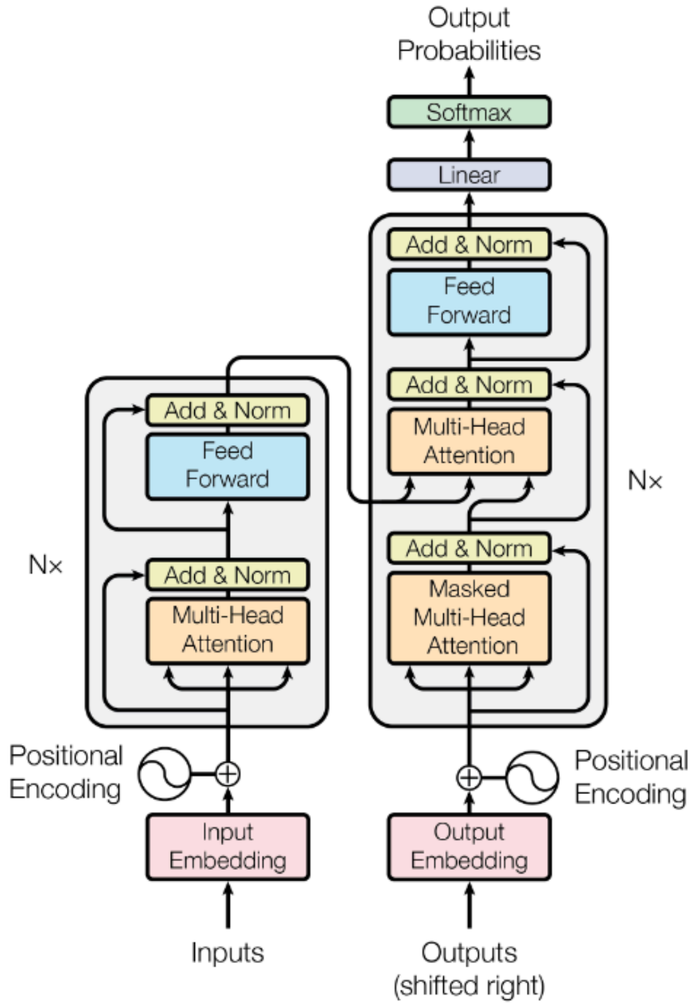
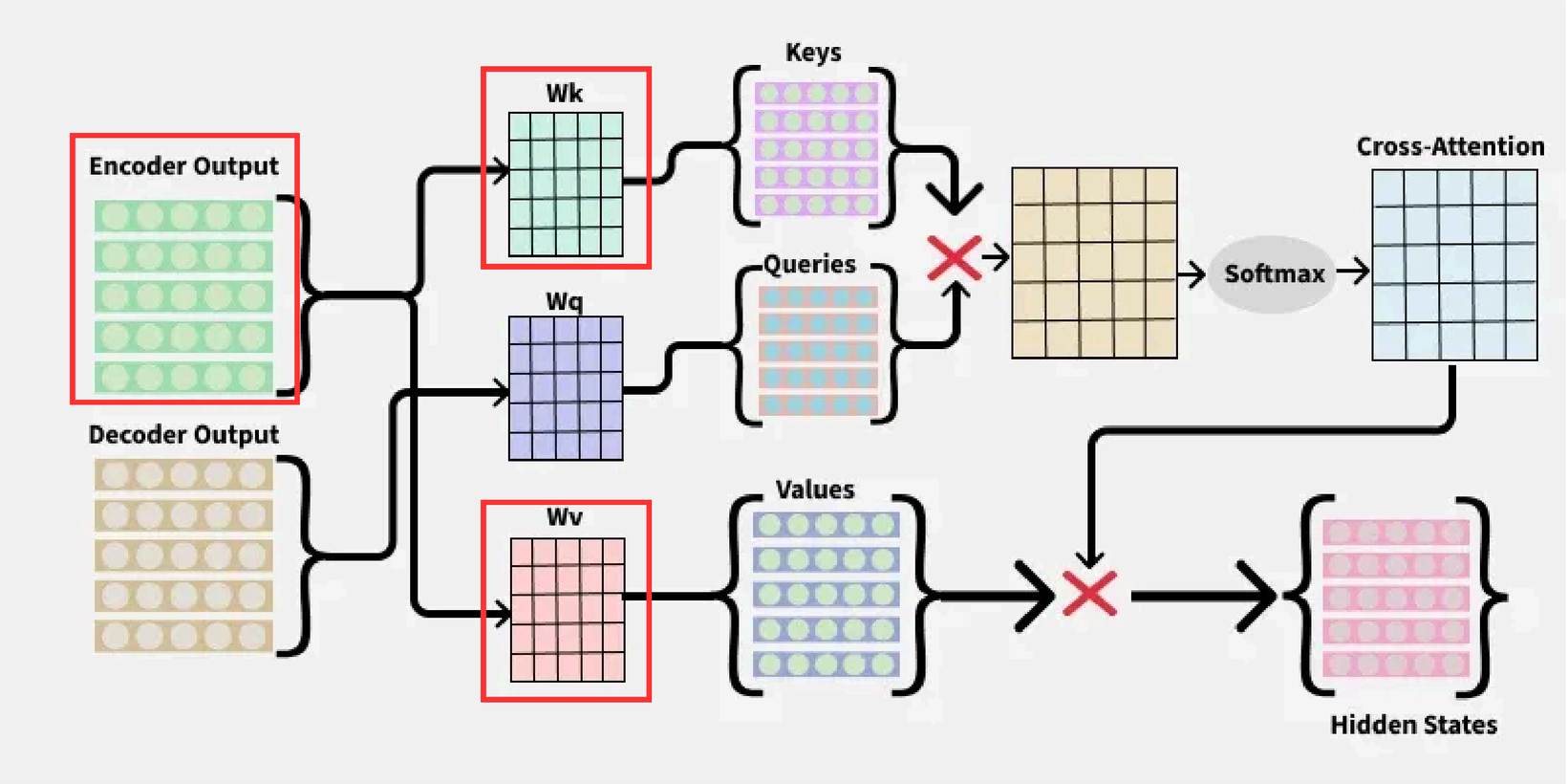


Figure 1: The Transformer - model architecture.

Cross Attention



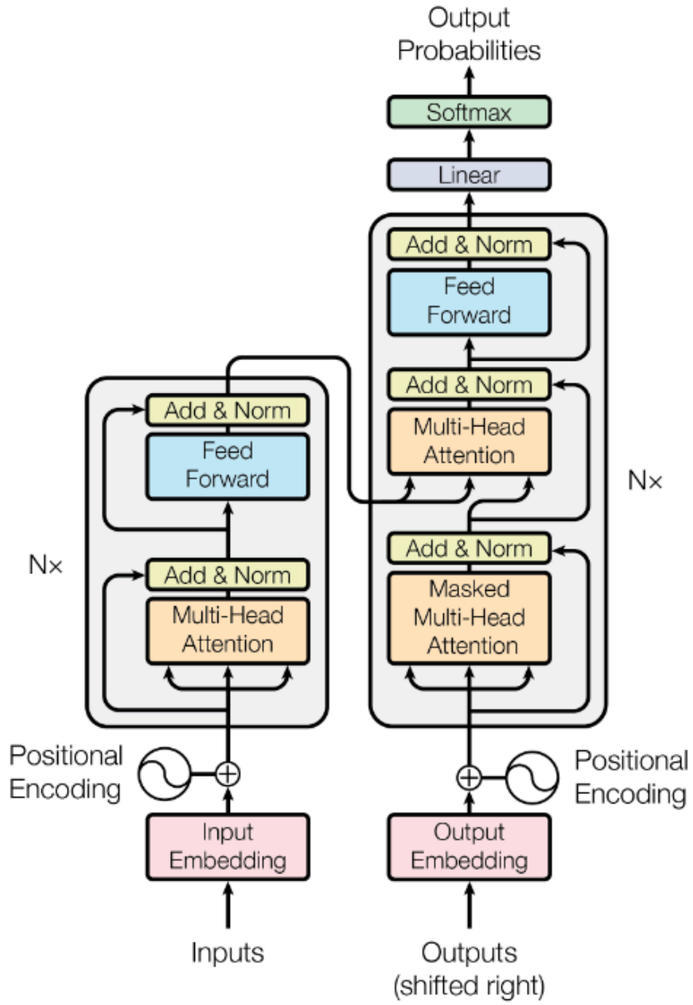
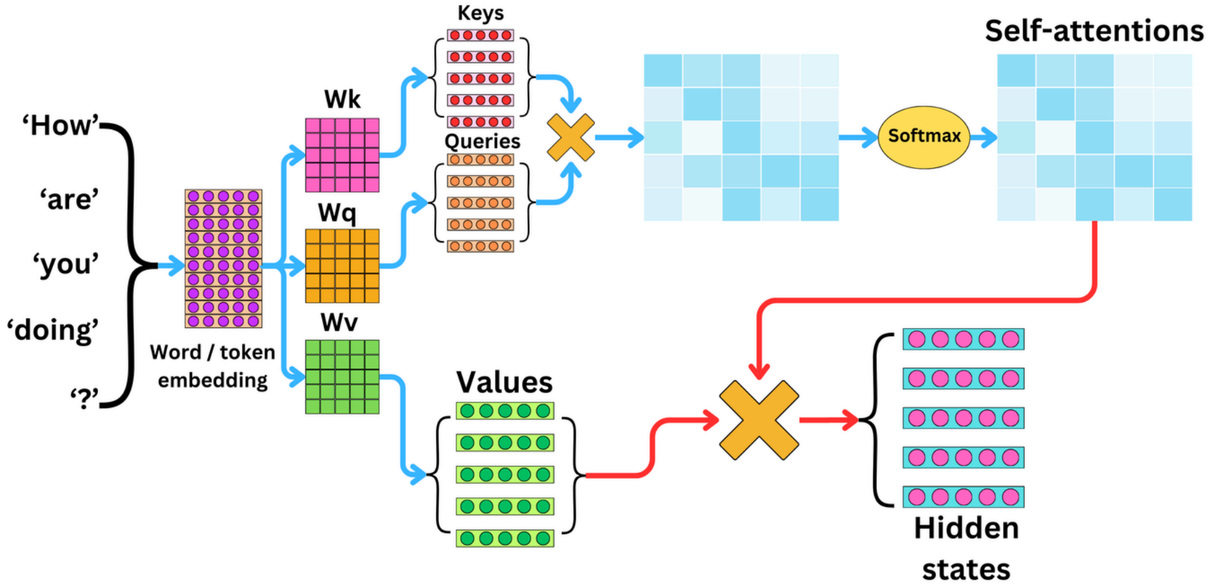
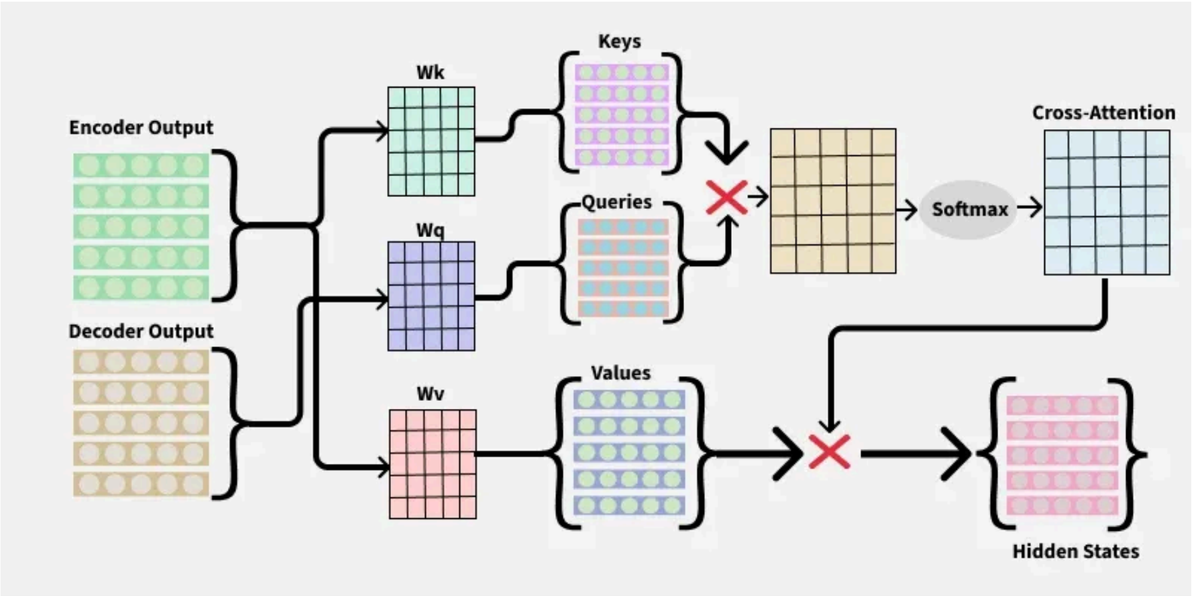


Figure 1: The Transformer - model architecture.

Cross-Attention vs Self-Attention



Each word looks at other words in the same sentence



Decoder looks at encoder output

Encoder

Prediction Head

Decoder

Feed Forward Layer

Layer Normalization

Multi-Head Attention!
(list of parallel self-attentions)

Residual Connection

Multi-Head Attention!
(list of parallel cross-attentions)

Masked Multi-Head Attention!
(list of parallel masked-attentions)

Embeddings
(Token and Positional)

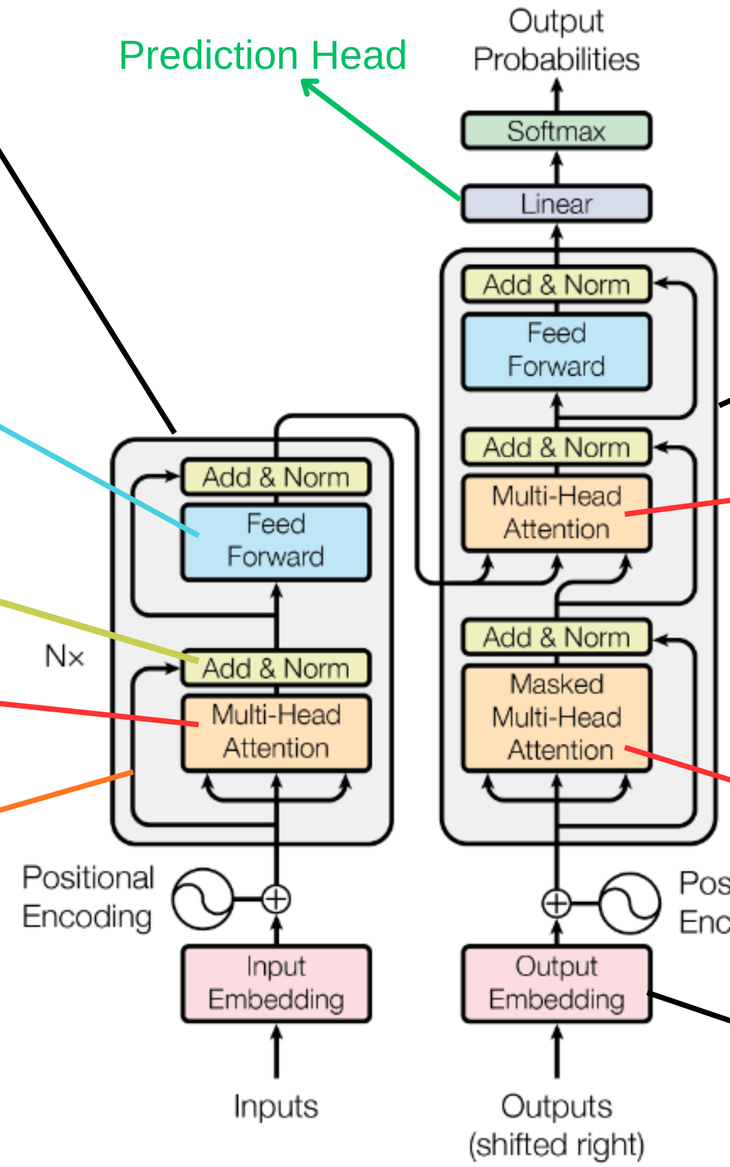


Figure 1: The Transformer - model architecture.

Nx: number of continously connected encoders/decoders

Masked Attention

$$M = \begin{bmatrix} 0 & -inf & -inf & -inf & -inf \\ 0 & 0 & -inf & -inf & -inf \\ 0 & 0 & 0 & -inf & -inf \\ 0 & 0 & 0 & 0 & -inf \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

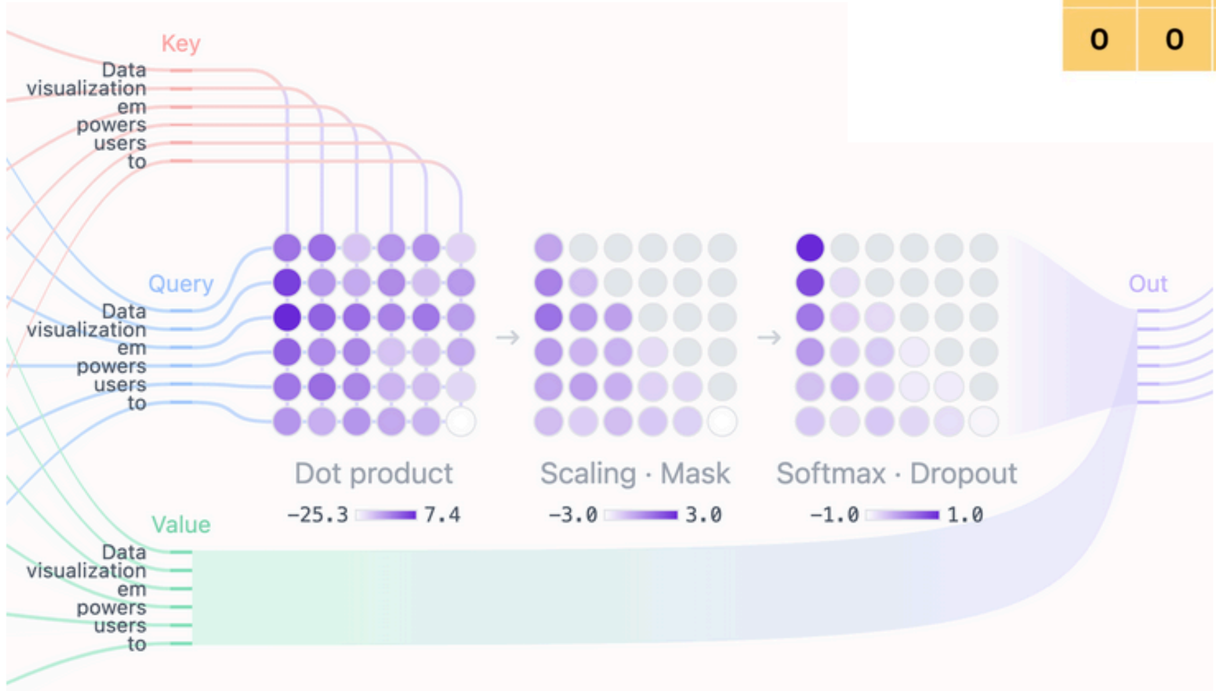


Figure 3. Using Query, Key, and Value matrices to calculate masked self-attention.

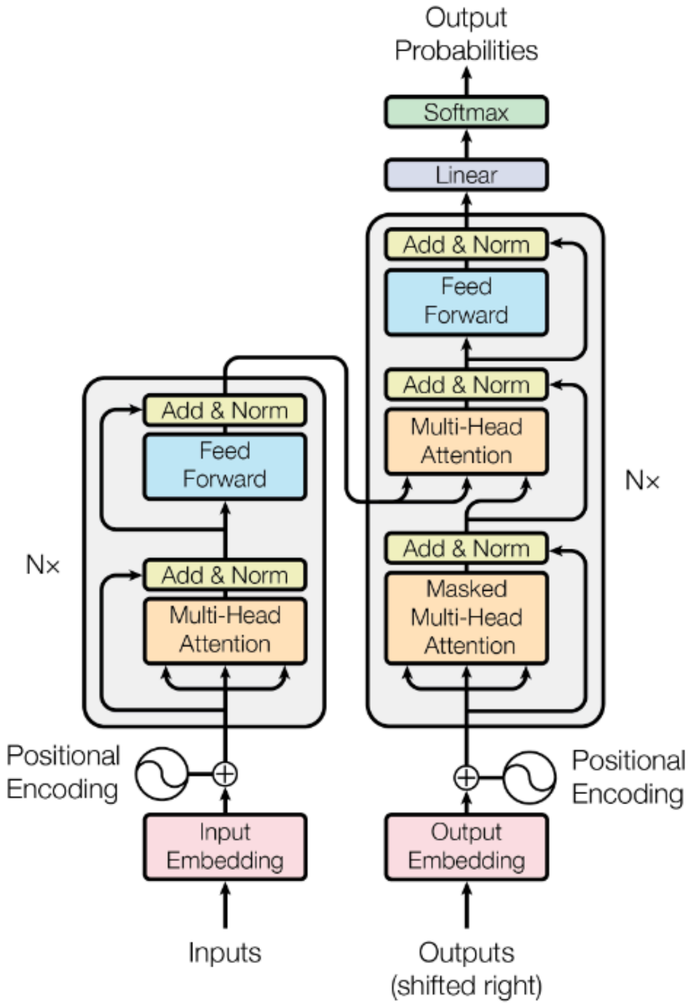


Figure 1: The Transformer - model architecture.

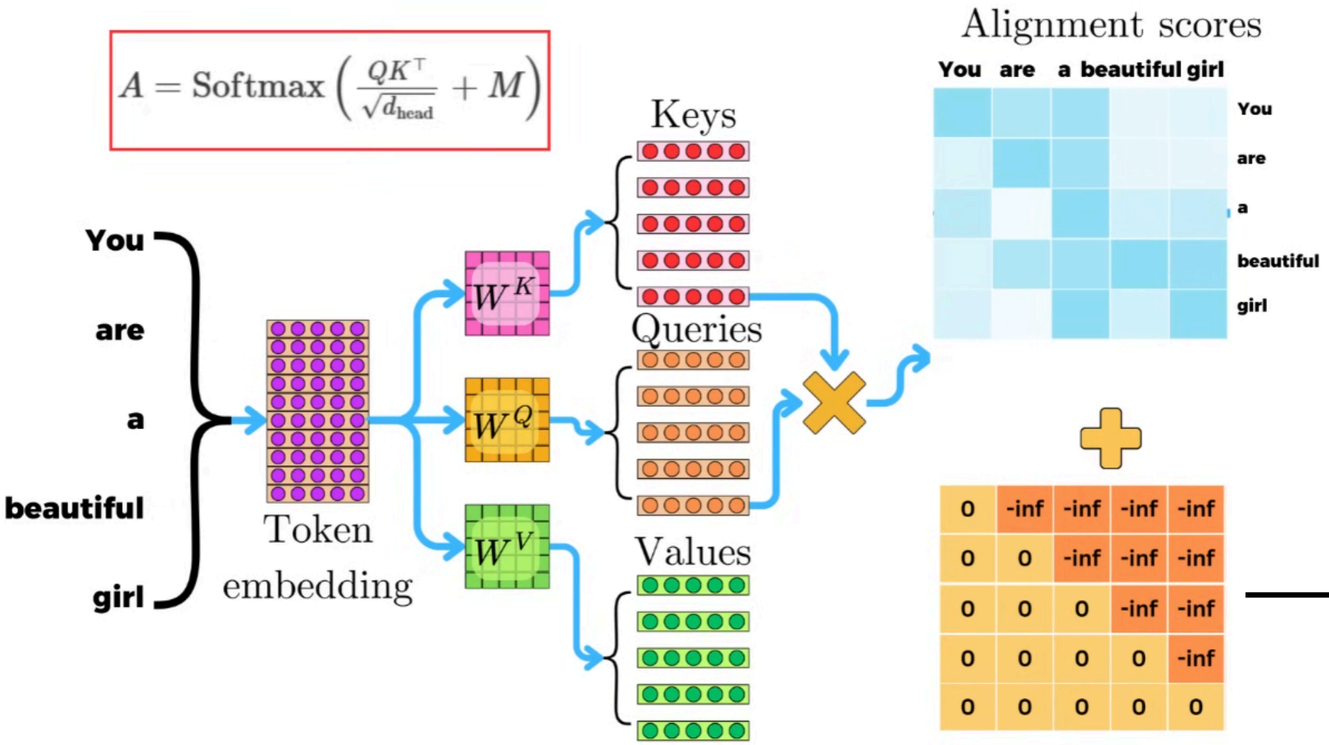
Masked Attention

(N x N) where N is number of tokens

$M =$

0	-inf	-inf	-inf	-inf
0	0	-inf	-inf	-inf
0	0	0	-inf	-inf
0	0	0	0	-inf
0	0	0	0	0

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M\right)$$

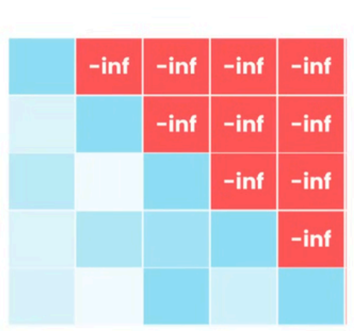


0	-inf	-inf	-inf	-inf
0	0	-inf	-inf	-inf
0	0	0	-inf	-inf
0	0	0	0	-inf
0	0	0	0	0

Attention mask

Block model accesses to future tokens.

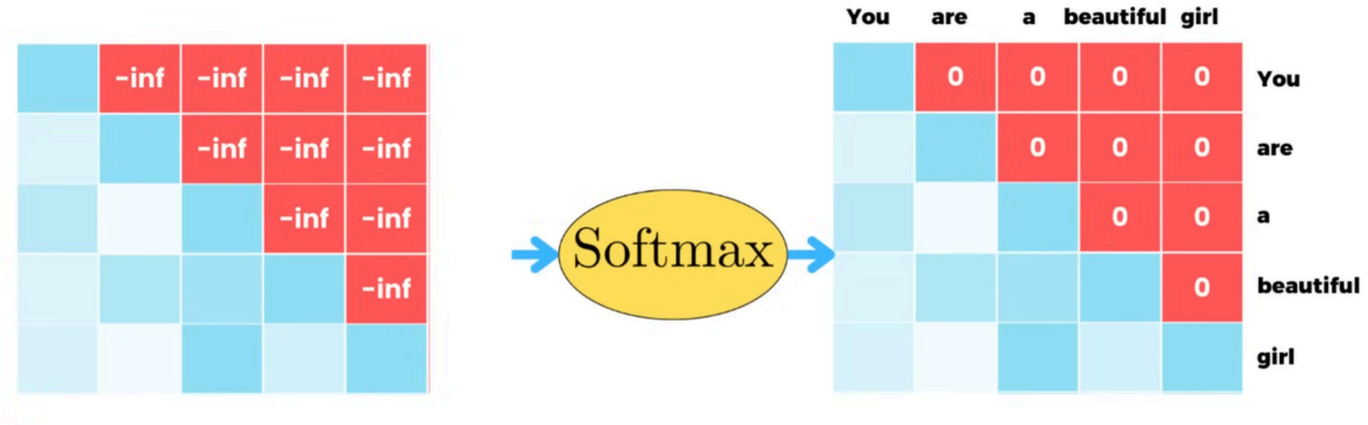
Prevent a token from seeing certain other tokens.





Why Masked Attention?

- Ensures causal, sequential prediction
- Makes the model generate one token at a time without cheating

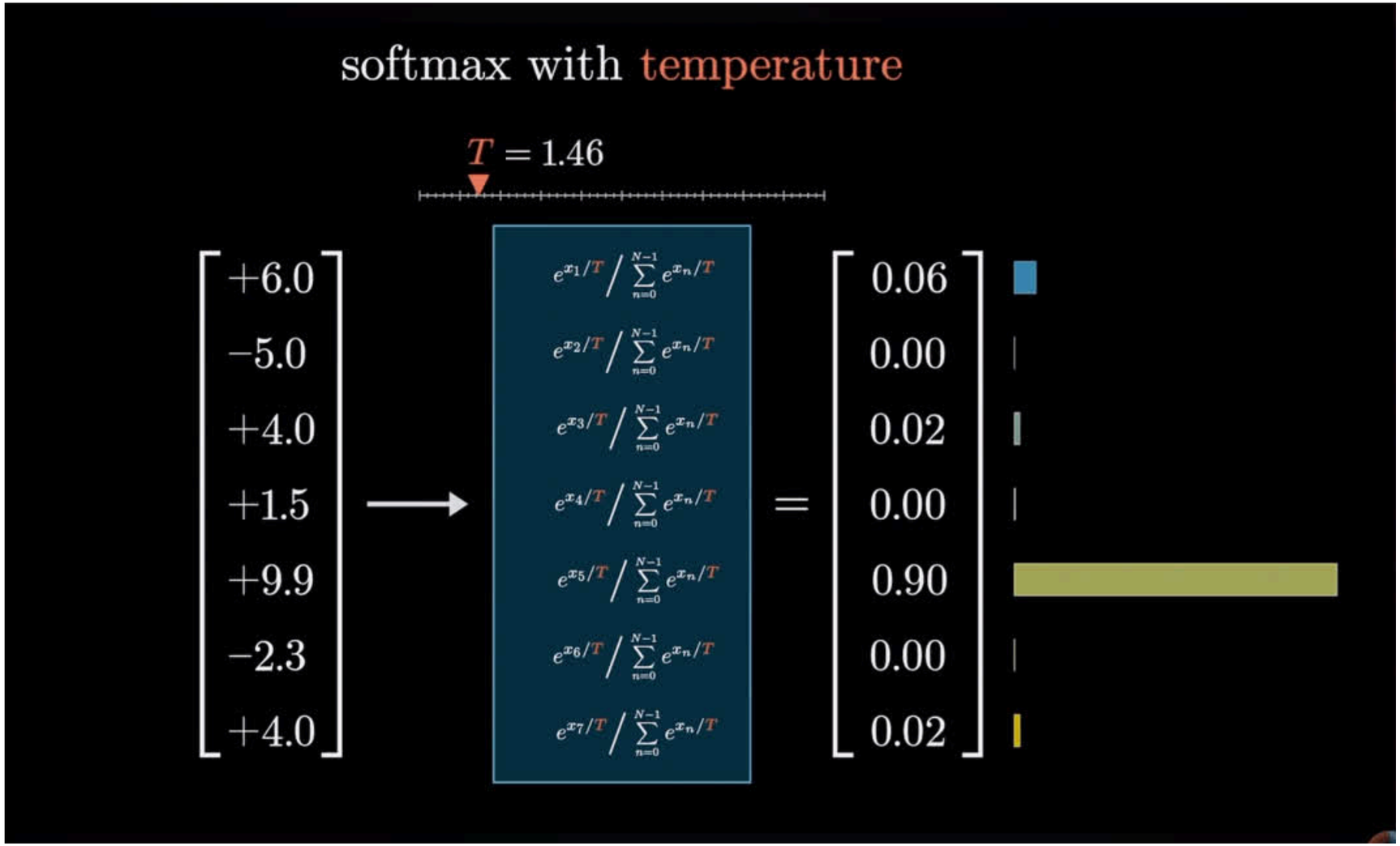


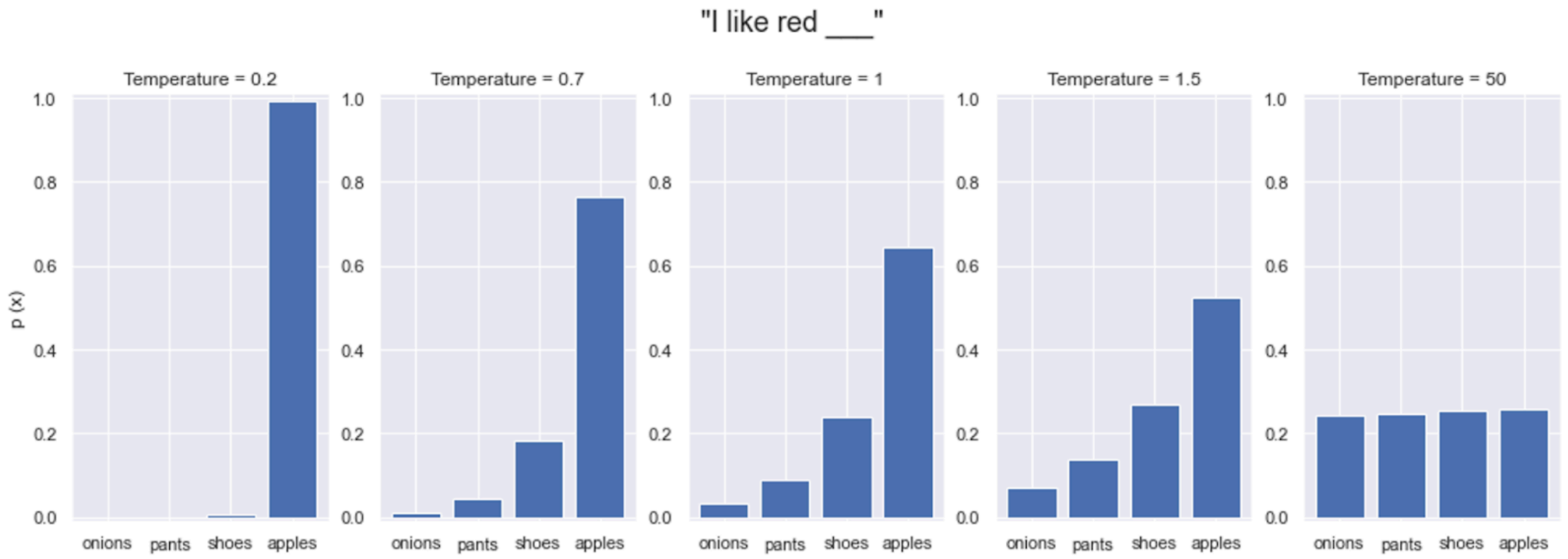
09

Temperature

Softmax function

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_n e^{x_n}} \quad \rightarrow \quad \text{softmax}(x_i) = \frac{e^{\frac{x_i}{T}}}{\sum_n e^{\frac{x_n}{T}}}$$


softmax with temperature
$T = 1.46$
$\begin{bmatrix} +6.0 \\ -5.0 \\ +4.0 \\ +1.5 \\ +9.9 \\ -2.3 \\ +4.0 \end{bmatrix}$
$e^{x_1/T} \Big/ \sum_{n=0}^{N-1} e^{x_n/T}$
$e^{x_2/T} \Big/ \sum_{n=0}^{N-1} e^{x_n/T}$
$e^{x_3/T} \Big/ \sum_{n=0}^{N-1} e^{x_n/T}$
$e^{x_4/T} \Big/ \sum_{n=0}^{N-1} e^{x_n/T}$
$e^{x_5/T} \Big/ \sum_{n=0}^{N-1} e^{x_n/T}$
$e^{x_6/T} \Big/ \sum_{n=0}^{N-1} e^{x_n/T}$
$e^{x_7/T} \Big/ \sum_{n=0}^{N-1} e^{x_n/T}$
$=$
$\begin{bmatrix} 0.06 \\ 0.00 \\ 0.02 \\ 0.00 \\ 0.90 \\ 0.00 \\ 0.02 \end{bmatrix}$


"I like red ___"
Temperature = 0.2
Temperature = 0.7
Temperature = 1
Temperature = 1.5
Temperature = 50
p (x)
1.0
0.8
0.6
0.4
0.2
0.0
onions
pants
shoes
apples

Thank you for listening!

